Basic instructions

8

8.1 Bit logic operations

8.1.1 Bit logic instructions

LAD and FBD are very effective for handling Boolean logic. While SCL is especially effective for complex mathematical computation and for project control structures, you can use SCL for Boolean logic.

LAD contacts

Table 8-1 Normally open and normally closed contacts

LAD	SCL	Description

"IN" 	`IF in THEN`
	`  Statement;`
	`ELSE`
	`  Statement;`
	`END_IF;`
		Normally open and normally closed contacts: You can connect contacts to other contacts and create your own combination logic. If the input bit you specify uses memory identifier I (input) or Q (output), then the bit value is read from the process-image register. The physical contact signals in your control process are wired to I terminals on the PLC. The CPU scans the wired input signals and continuously updates the corresponding state values in the process-image input register.
		You can perform an immediate read of a physical input using ":P" following the I offset (example: "%I3.4:P"). For an immediate read, the bit data values are read directly from the physical input instead of the process image. An immediate read does not update the process image.

"IN" 	`IF NOT (in) THEN`
	`  Statement;`
	`ELSE`
	`  Statement;`
	`END_IF;`	

Table 8-2 Data types for the parameters

Parameter	Data type	Description
IN	Bool	Assigned bit

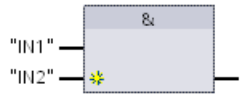
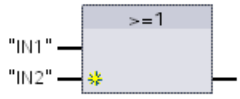
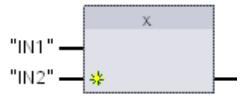
- The Normally Open contact is closed (ON) when the assigned bit value is equal to 1.
- The Normally Closed contact is closed (ON) when the assigned bit value is equal to 0.
- Contacts connected in series create AND logic networks.
- Contacts connected in parallel create OR logic networks.

FBD AND, OR, and XOR boxes

In FBD programming, LAD contact networks are transformed into AND (&), OR (>=1), and EXCLUSIVE OR (x) box networks where you can specify bit values for the box inputs and outputs. You may also connect to other logic boxes and create your own logic combinations. After the box is placed in your network, you can drag the "Insert input" tool from the "Favorites" toolbar or instruction tree and then drop it onto the input side of the box to add more inputs. You can also right-click on the box input connector and select "Insert input".

Box inputs and outputs can be connected to another logic box, or you can enter a bit address or bit symbol name for an unconnected input. When the box instruction is executed, the current input states are applied to the binary box logic and, if true, the box output will be true.

Table 8-3 AND, OR, and XOR boxes

FBD	SCL[1]	Description
	`out := in1 AND in2;`	All inputs of an AND box must be TRUE for the output to be TRUE.
	`out := in1 OR in2;`	Any input of an OR box must be TRUE for the output to be TRUE.
	`out := in1 XOR in2;`	An odd number of the inputs of an XOR box must be TRUE for the output to be TRUE.

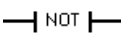
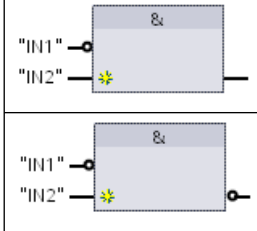
[1] For SCL: You must assign the result of the operation to a variable to be used for another statement.

Table 8-4 Data types for the parameters

Parameter	Data type	Description
IN1, IN2	Bool	Input bit

NOT logic inverter

Table 8-5 Invert RLO (Result of Logic Operation)

LAD	FBD	SCL	Description
		**NOT**	For FBD programming, you can drag the "Invert RLO" tool from the "Favorites" toolbar or instruction tree and then drop it on an input or output to create a logic inverter on that box connector. The LAD NOT contact inverts the logical state of power flow input. • If there is no power flow into the NOT contact, then there is power flow out. • If there is power flow into the NOT contact, then there is no power flow out.

Output coil and assignment box

The coil output instruction writes a value for an output bit. If the output bit you specify uses memory identifier Q, then the CPU turns the output bit in the process-image register on or off, setting the specified bit equal to power flow status. The output signals for your control actuators are wired to the Q terminals of the CPU. In RUN mode, the CPU system continuously scans your input signals, processes the input states according to your program logic, and then reacts by setting new output state values in the process-image output register. The CPU system transfers the new output state reaction that is stored in the process-image register, to the wired output terminals.

Table 8-6 Assignment and negate assignment

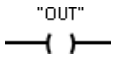
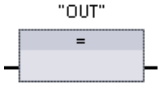
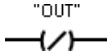
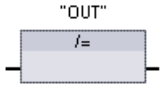
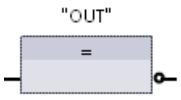
LAD	FBD	SCL	Description
		`out := <Boolean expression>;`	In FBD programming, LAD coils are transformed into assignment (= and /=) boxes where you specify a bit address for the box output. Box inputs and outputs can be connected to other box logic or you can enter a bit address. You can specify an immediate write of a physical output using ":P" following the Q offset (example: "%Q3.4:P"). For an immediate write, the bit data values are written to the process image output and directly to physical output.
		`out := NOT <Boolean expression>;`	
			

Table 8-7 Data types for the parameters

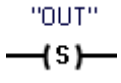
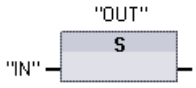
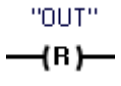
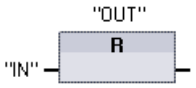
Parameter	Data type	Description
OUT	Bool	Assigned bit

- If there is power flow through an output coil or an FBD "=" box is enabled, then the output bit is set to 1.
- If there is no power flow through an output coil or an FBD "=" assignment box is not enabled, then the output bit is set to 0.
- If there is power flow through an inverted output coil or an FBD "/=" box is enabled, then the output bit is set to 0.
- If there is no power flow through an inverted output coil or an FBD "/=" box is not enabled, then the output bit is set to 1.

8.1.2 Set and reset instructions

Set and Reset 1 bit

Table 8-8 S and R instructions

LAD	FBD	SCL	Description
		Not available	Set output: When S (Set) is activated, then the data value at the OUT address is set to 1. When S is not activated, OUT is not changed.
		Not available	Reset output: When R (Reset) is activated, then the data value at the OUT address is set to 0. When R is not activated, OUT is not changed.

[1] For LAD and FBD: These instructions can be placed anywhere in the network.

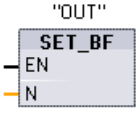
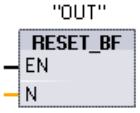
[2] For SCL: You must write code to replicate this function within your application.

Table 8-9 Data types for the parameters

Parameter	Data type	Description
IN (or connect to contact/gate logic)	Bool	Bit tag of location to be monitored
OUT	Bool	Bit tag of location to be set or reset

Set and Reset Bit Field

Table 8-10 SET_BF and RESET_BF instructions

LAD[1]	FBD	SCL	Description
		Not available	Set bit field: When SET_BF is activated, a data value of 1 is assigned to "n" bits starting at address tag OUT. When SET_BF is not activated, OUT is not changed.
		Not available	Reset bit field: RESET_BF writes a data value of 0 to "n" bits starting at address tag OUT. When RESET_BF is not activated, OUT is not changed.

[1] For LAD and FBD: These instructions must be the right-most instruction in a branch.

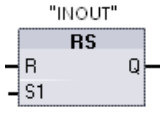
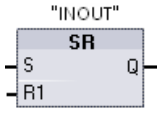
[2] For SCL: You must write code to replicate this function within your application.

Table 8-11 Data types for the parameters

Parameter	Data type	Description
OUT	Bool	Starting element of a bit field to be set or reset (Example: #MyArray[3])
n	Constant (UInt)	Number of bits to write

Set-dominant and Reset-dominant flip-flops

Table 8-12 RS and SR instructions

LAD / FBD	SCL	Description
	Not available	Reset/set flip-flop: RS is a set dominant latch where the set dominates. If the set (S1) and reset (R) signals are both true, the value at address INOUT will be 1.
	Not available	Set/reset flip-flop: SR is a reset dominant latch where the reset dominates. If the set (S) and reset (R1) signals are both true, the value at address INOUT will be 0.

[1] For LAD and FBD: These instructions must be the right-most instruction in a branch.

[2] For SCL: You must write code to replicate this function within your application.

Table 8-13 Data types for the parameters

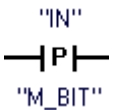
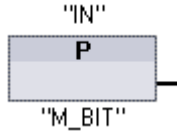
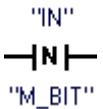
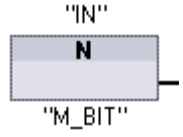
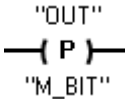
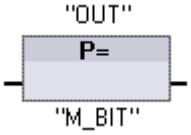
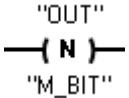
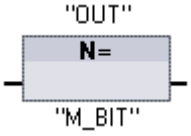
Parameter	Data type	Description
S, S1	Bool	Set input; 1 indicates dominance
R, R1	Bool	Reset input; 1 indicates dominance
INOUT	Bool	Assigned bit tag "INOUT"
Q	Bool	Follows state of "INOUT" bit

The "INOUT" tag assigns the bit address that is set or reset. The optional output Q follows the signal state of the "INOUT" address.

Instruction	S1	R	"INOUT" bit
RS	0	0	Previous state
	0	1	0
	1	0	1
	1	1	1
	S	**R1**	
SR	0	0	Previous state
	0	1	0
	1	0	1
	1	1	0

8.1.3 Positive and negative edge instructions

Table 8-14 Positive and negative transition detection

LAD	FBD	SCL	Description
		Not available [1]	Scan operand for positive signal edge. LAD: The state of this contact is TRUE when a positive transition (OFF-to-ON) is detected on the assigned "IN" bit. The contact logic state is then combined with the power flow in state to set the power flow out state. The P contact can be located anywhere in the network except the end of a branch. FBD: The output logic state is TRUE when a positive transition (OFF-to-ON) is detected on the assigned input bit. The P box can only be located at the beginning of a branch.
		Not available [1]	Scan operand for negative signal edge. LAD: The state of this contact is TRUE when a negative transition (ON-to-OFF) is detected on the assigned input bit. The contact logic state is then combined with the power flow in state to set the power flow out state. The N contact can be located anywhere in the network except the end of a branch. FBD: The output logic state is TRUE when a negative transition (ON-to-OFF) is detected on the assigned input bit. The N box can only be located at the beginning of a branch.
		Not available [1]	Set operand on positve signal edge. LAD: The assigned bit "OUT" is TRUE when a positive transition (OFF-to-ON) is detected on the power flow entering the coil. The power flow in state always passes through the coil as the power flow out state. The P coil can be located anywhere in the network. FBD: The assigned bit "OUT" is TRUE when a positive transition (OFF-to-ON) is detected on the logic state at the box input connection or on the input bit assignment if the box is located at the start of a branch. The input logic state always passes through the box as the output logic state. The P= box can be located anywhere in the branch.
		Not available [1]	Set operand on negative signal edge. LAD: The assigned bit "OUT" is TRUE when a negative transition (ON-to-OFF) is detected on the power flow entering the coil. The power flow in state always passes through the coil as the power flow out state. The N coil can be located anywhere in the network. FBD: The assigned bit "OUT" is TRUE when a negative transition (ON-to-OFF) is detected on the logic state at the box input connection or on the input bit assignment if the box is located at the start of a branch. The input logic state always passes through the box as the output logic state. The N= box can be located anywhere in the branch.

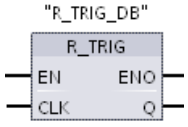
[1] For SCL: You must write code to replicate this function within your application.

Table 8-15 P_TRIG and N_TRIG

LAD / FBD	SCL	Description
	Not available [1]	Scan RLO (result of logic operation) for positve signal edge. The Q output power flow or logic state is TRUE when a positive transition (OFF-to-ON) is detected on the CLK input state (FBD) or CLK power flow in (LAD). In LAD, the P_TRIG instruction cannot be located at the beginning or end of a network. In FBD, the P_TRIG instruction can be located anywhere except the end of a branch.
	Not available [1]	Scan RLO for negative signal edge. The Q output power flow or logic state is TRUE when a negative transition (ON-to-OFF) is detected on the CLK input state (FBD) or CLK power flow in (LAD). In LAD, the N_TRIG instruction cannot be located at the beginning or end of a network. In FBD, the N_TRIG instruction can be located anywhere except the end of a branch.

[1] For SCL: You must write code to replicate this function within your application.

Table 8-16 R_TRIG and F_TRIG instructions

LAD / FBD	SCL	Description
	**"R_TRIG_DB"(CLK:=_in_, Q=> _bool_out_);**	Set tag on positive signal edge. The assigned instance DB is used to store the previous state of the CLK input. The Q output power flow or logic state is TRUE when a positive transition (OFF-to-ON) is detected on the CLK input state (FBD) or CLK power flow in (LAD). In LAD, the R_TRIG instruction cannot be located at the beginning or end of a network. In FBD, the R_TRIG instruction can be located anywhere except the end of a branch.
	"F_TRIG_DB"(CLK:=_in_, Q=> _bool_out_);	Set tag on negative signal edge. The assigned instance DB is used to store the previous state of the CLK input. The Q output power flow or logic state is TRUE when a negative transition (ON-to-OFF) is detected on the CLK input state (FBD) or CLK power flow in (LAD). In LAD, the F_TRIG instruction cannot be located at the beginning or end of a network. In FBD, the F_TRIG instruction can be located anywhere except the end of a branch.

For R_TRIG and F_TRIG, when you insert the instruction in the program, the "Call options" dialog opens automatically. In this dialog you can assign
whether the edge memory bit is stored in its own data block (single instance) or as a local tag (multiple instance) in the
block interface. If you create a separate data block, you will find it in the project tree in the

"Program resources" folder
under "Program blocks > System blocks".

Table 8-17 Data types for the parameters (P and N contacts/coils, P=, N=, P_TRIG and N_TRIG)

Parameter	Data type	Description
M_BIT	Bool	Memory bit in which the previous state of the input is saved
IN	Bool	Input bit whose transition edge is detected
OUT	Bool	Output bit which indicates a transition edge was detected
CLK	Bool	Power flow or input bit whose transition edge is detected
Q	Bool	Output which indicates an edge was detected

All edge instructions use a memory bit (M_BIT: P/N contacts/coils, P_TRIG/N_TRIG) or (instance DB bit: R_TRIG, F_TRIG) to store the previous state of the monitored input signal. An edge is detected by comparing the state of the input with the previous state. If the states indicate a change of the input in the direction of interest, then an edge is reported by writing the output TRUE. Otherwise, the output is written to FALSE.

Note

Edge instructions evaluate the input and memory-bit values each time they are executed, including the first execution. You must account for the initial states of the input and memory bit in your program design either to allow or to avoid edge detection on the first scan.

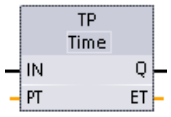
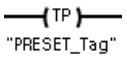
Because the memory bit must be maintained from one execution to the next, you should use a unique bit for each edge instruction, and you should not use this bit any other place in your program. You should also avoid temporary memory and memory that can be affected by other system functions, such as an I/O update. Use only M, global DB, or Static memory (in an instance DB) for M_BIT memory assignments.

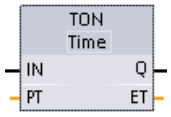
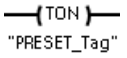
8.2 Timer operations

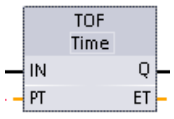
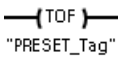
You use the timer instructions to create programmed time delays. The number of timers that you can use in your user program is limited only by the amount of memory in the CPU. Each timer uses a 16 byte IEC_Timer data type DB structure to store timer data that is specified at the top of the box or coil instruction. STEP 7 automatically creates the DB when you insert the instruction.

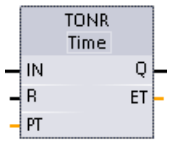
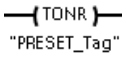
Table 8-18 Timer instructions

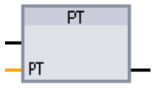
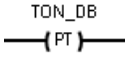
LAD / FBD boxes	LAD coils	SCL	Description

IEC_Timer_0
 
	TP_DB
	 
		`"IEC_Timer_0_DB".TP(`
		`IN:=_bool_in_,`
		`PT:=_time_in_,`
		`Q=>_bool_out_,`
		`ET=>_time_out_);`
			The TP timer generates a pulse with a preset width time.

IEC_Timer_1
 
	TON_DB
	 
		`"IEC_Timer_0_DB".TON (`
		`IN:=_bool_in_,`
		`PT:=_time_in_,`
		`Q=>_bool_out_,`
		`ET=>_time_out_);`
			The TON timer sets output Q to ON after a preset time delay.

IEC_Timer_2
 
	TOF_DB
	 
		`"IEC_Timer_0_DB".TOF (`
		`IN:=_bool_in_,`
		`PT:=_time_in_,`
		`Q=>_bool_out_,`
		`ET=>_time_out_);`
			The TOF timer resets output Q to OFF after a preset time delay.

IEC_Timer_3
 
	TONR_DB
	 
		`"IEC_Timer_0_DB".TONR (`
		`IN:=_bool_in_,`
		`R:=_bool_in_,`
		`PT:=_time_in_,`
		`Q=>_bool_out_,`
		`ET=>_time_out_);`
			The TONR timer sets output Q to ON after a preset time delay. Elapsed time is accumulated over multiple timing periods until the R input is used to reset the elapsed time.

FBD only:
 
	TON_DB
	 
		`PRESET_TIMER(`
		`PT:=_time_in_,`
		`TIMER:=_iec_timer_in_);`
			The PT (Preset timer) coil loads a new PRESET time value in the specified IEC_Timer.

FBD only: 	TON_DB 	`RESET_TIMER(` `_iec_timer_in_);`	The RT (Reset timer) coil resets the specified IEC_Timer.

[1] STEP 7 automatically creates the DB when you insert the instruction.

[2] In the SCL examples, "IEC_Timer_0_DB" is the name of the instance DB.

Table 8-19 Data types for the parameters

Parameter	Data type	Description
Box: IN Coil: Power flow	Bool	TP, TON, and TONR: Box: 0=Disable timer, 1=Enable timer Coil: No power flow=Disable timer, Power flow=Enable timer TOF: Box: 0=Enable timer, 1=Disable timer Coil: No power flow=Enable timer, Power flow=Disable timer
R	Bool	TONR box only: 0=No reset 1= Reset elapsed time and Q bit to 0
Box: PT Coil: "PRESET_Tag"	Time	Timer box or coil: Preset time input
Box: Q Coil: DBdata.Q	Bool	Timer box: Q box output or Q bit in the timer DB data Timer coil: you can only address the Q bit in the timer DB data
Box: ET Coil: DBdata.ET	Time	Timer box: ET (elapsed time) box output or ET time value in the timer DB data Timer coil: you can only address the ET time value in the timer DB data.

Table 8-20 Effect of value changes in the PT and IN parameters

Timer	Changes in the PT and IN box parameters and the corresponding coil parameters
TP	• Changing PT has no effect while the timer runs. • Changing IN has no effect while the timer runs.
TON	• Changing PT has no effect while the timer runs. • Changing IN to FALSE, while the timer runs, resets and stops the timer.
TOF	• Changing PT has no effect while the timer runs. • Changing IN to TRUE, while the timer runs, resets and stops the timer.
TONR	• Changing PT has no effect while the timer runs, but has an effect when the timer resumes. • Changing IN to FALSE, while the timer runs, stops the timer but does not reset the timer. Changing IN back to TRUE will cause the timer to start timing from the accumulated time value.

PT (preset time) and ET (elapsed time) values are stored in the specified IEC_TIMER DB data as signed double integers that represent milliseconds of time. TIME data uses the T# identifier and can be entered as a simple time unit (T#200ms or 200) and as compound time units like T#2s_200ms.

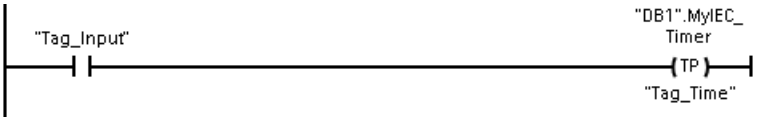
Table 8-21 Size and range of the TIME data type

Data type	Size	Valid number ranges[1]
TIME	32 bits, stored as DInt data	T#-24d_20h_31m_23s_648ms to T#24d_20h_31m_23s_647ms Stored as -2,147,483,648 ms to +2,147,483,647 ms

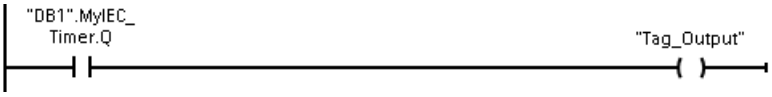
[1] The negative range of the TIME data type shown above cannot be used with the timer instructions. Negative PT (preset time) values are set to zero when the timer instruction is executed. ET (elapsed time) is always a positive value.

Timer coil example

The -(TP)-, -(TON)-, -(TOF)-, and -(TONR)- timer coils must be the last instruction in a LAD network. As shown in the timer example, a contact instruction in a subsequent network evaluates the Q bit in a timer coil's IEC_Timer DB data. Likewise, you must address the ELAPSED element in the IEC_timer DB data if you want to use the elapsed time value in your program.



The pulse timer is started on a 0 to 1 transition of the Tag_Input bit value. The timer runs for the time specified by Tag_Time time value.



As long as the timer runs, the state of DB1.MyIEC_Timer.Q=1 and the Tag_Output value=1. When the Tag_Time value has elapsed, then DB1.MyIEC_Timer.Q=0 and the Tag_Output value=0.

Reset timer -(RT)- and Preset timer -(PT)- coils

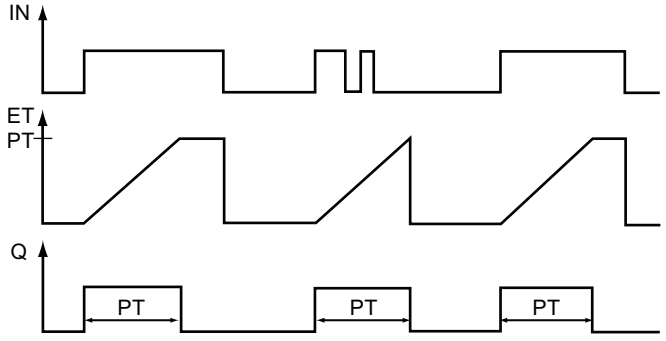
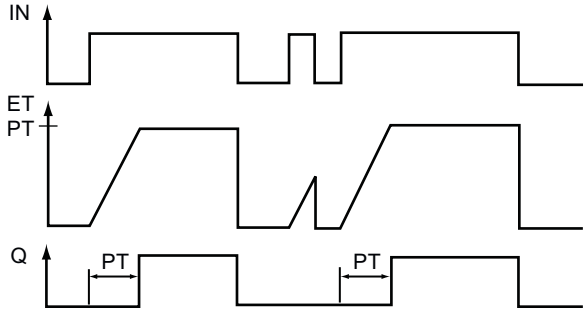
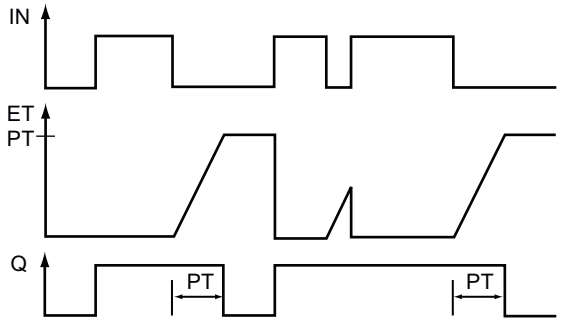
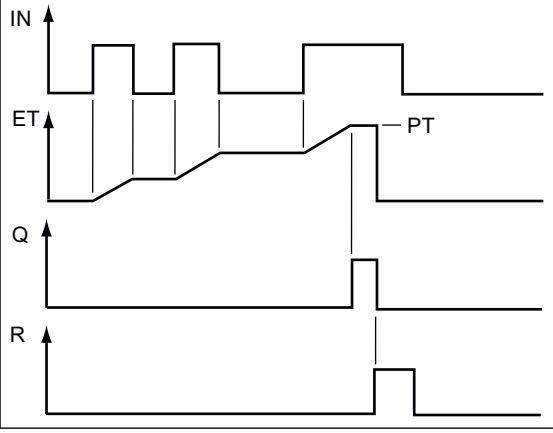
These coil instructions can be used with box or coil timers and can be placed in a mid-line position. The coil output power flow status is always the same as the coil input status. When the -(RT)- coil is activated, the ELAPSED time element of the specified IEC_Timer DB data is reset to 0. When the -(PT)- coil is activated, the PRESET time element of the specified IEC_Timer DB data is loaded with the assigned time-duration value..

Note

When you place timer instructions in an FB, you can select the "Multi-instance data block" option. The timer structure names can be different with separate data structures, but the timer data is contained in a single data block and does not require a separate data block for each timer. This reduces the processing time and data storage necessary for handling the timers. There is no interaction between the timer data structures in the shared multi-instance DB.

Operation of the timers

Table 8-22 Types of IEC timers

Timer	Timing diagram
TP: Generate pulse The TP timer generates a pulse with a preset width time.	
TON: Generate ON-delay The TON timer sets output Q to ON after a preset time delay.	
TOF: Generate OFF-delay The TOF timer resets output Q to OFF after a preset time delay.	
TONR: Time accumulator The TONR timer sets output Q to ON after a preset time delay. Elapsed time is accumulated over multiple timing periods until the R input is used to reset the elapsed time.	

Note

In the CPU, no dedicated resource is allocated to any specific timer instruction. Instead, each timer utilizes its own timer structure in DB memory and a continuously-running internal CPU timer to perform timing.

When a timer is started due to an edge change on the input of a TP, TON, TOF, or TONR instruction, the value of the continuously-running internal CPU timer is copied into the START member of the DB structure allocated for this timer instruction. This start value remains unchanged while the timer continues to run, and is used later each time the timer is updated. Each time the timer is started, a new start value is loaded into the timer structure from the internal CPU timer.

When a timer is updated, the start value described above is subtracted from the current value of the internal CPU timer to determine the elapsed time. The elapsed time is then compared with the preset to determine the state of the timer Q bit. The ELAPSED and Q members are then updated in the DB structure allocated for this timer. Note that the elapsed time is clamped at the preset value (the timer does not continue to accumulate elapsed time after the preset is reached).

A timer update is performed when and only when:

- A timer instruction (TP, TON, TOF, or TONR) is executed
- The "ELAPSED" member of the timer structure in DB is referenced directly by an instruction
- The "Q" member of the timer structure in DB is referenced directly by an instruction

Timer programming

The following consequences of timer operation should be considered when planning and creating your user program:

- You can have multiple updates of a timer in the same scan. The timer is updated each time the timer instruction (TP, TON, TOF, TONR) is executed and each time the ELAPSED or Q member of the timer structure is used as a parameter of another executed instruction. This is an advantage if you want the latest time data (essentially an immediate read of the timer). However, if you desire to have consistent values throughout a program scan, then place your timer instruction prior to all other instructions that need these values, and use tags from the Q and ET outputs of the timer instruction instead of the ELAPSED and Q members of the timer DB structure.
- You can have scans during which no update of a timer occurs. It is possible to start your timer in a function, and then cease to call that function again for one or more scans. If no other instructions are executed which reference the ELAPSED or Q members of the timer structure, then the timer will not be updated. A new update will not occur until either the timer instruction is executed again or some other instruction is executed using ELAPSED or Q from the timer structure as a parameter.

- Although not typical, you can assign the same DB timer structure to multiple timer instructions. In general, to avoid unexpected interaction, you should only use one timer instruction (TP, TON, TOF, TONR) per DB timer structure.
- Self-resetting timers are useful to trigger actions that need to occur periodically. Typically, self-resetting timers are created by placing a normally-closed contact which references the timer bit in front of the timer instruction. This timer network is typically located above one or more dependent networks that use the timer bit to trigger actions. When the timer expires (elapsed time reaches preset value), the timer bit is ON for one scan, allowing the dependent network logic controlled by the timer bit to execute. Upon the next execution of the timer network, the normally closed contact is OFF, thus resetting the timer and clearing the timer bit. The next scan, the normally closed contact is ON, thus restarting the timer. When creating self-resetting timers such as this, do not use the "Q" member of the timer DB structure as the parameter for the normally-closed contact in front of the timer instruction. Instead, use the tag connected to the "Q" output of the timer instruction for this purpose. The reason to avoid accessing the Q member of the timer DB structure is because this causes an update to the timer and if the timer is updated due to the normally closed contact, then the contact will reset the timer instruction immediately. The Q output of the timer instruction will not be ON for the one scan and the dependent networks will not execute.

Time data retention after a RUN-STOP-RUN transition or a CPU power cycle

If a run mode session is ended with stop mode or a CPU power cycle and a new run mode session is started, then the timer data stored in the previous run mode session is lost, unless the timer data structure is specified as retentive (TP, TON, TOF, and TONR timers).

When you accept the defaults in the call options dialog after you place a timer instruction in the program editor, you are automatically assigned an instance DB which **cannot be made retentive**. To make your timer data retentive, you must either use a global DB or a Multi-instance DB.

Assign a global DB to store timer data as retentive data

This option works regardless of where the timer is placed (OB, FC, or FB).

1. Create a global DB:
   - Double-click "Add new block" from the Project tree
   - Click the data block (DB) icon
   - For the Type, choose global DB
   - If you want to be able to select individual data elements in this DB as retentive, be sure the DB type "Optimized" box is checked. The other DB type option "Standard - compatible with S7-300/400" only allows setting all DB data elements retentive or none retentive.
   - Click OK
2. Add timer structure(s) to the DB:
   - In the new global DB, add a new static tag using data type IEC_Timer.
   - In the "Retain" column, check the box so that this structure will be retentive.
   - Repeat this process to create structures for all the timers that you want to store in this DB. You can either place each timer structure in a unique global DB, or you can place multiple timer structures into the same global DB. You can also place other static tags besides timers in this global DB. Placing multiple timer structures into the same global DB allows you to reduce your overall number of blocks.
   - Rename the timer structures if desired.
3. Open the program block for editing where you want to place a retentive timer (OB, FC, or FB).
4. Place the timer instruction at the desired location.
5. When the call options dialog appears, click the cancel button.
6. On the top of the new timer instruction, type the name (do not use the helper to browse) of the global DB and timer structure that you created above (example: "Data_block_3.Static_1").

Assign a multi-instance DB to store timer data as retentive data

This option only works if you place the timer in an FB.

This option depends upon whether the FB properties specify "Optimized block access" (allows symbolic access only). To verify how the access attribute is configured for an existing FB, right-click on the FB in the Project tree, choose properties, and then choose Attributes.

If the FB specifies "Optimized block access" (allows symbolic access only):

1. Open the FB for edit.
2. Place the timer instruction at the desired location in the FB.
3. When the Call options dialog appears, click the Multi instance icon. The Multi Instance option is only available if the instruction is being placed into an FB.
4. In the Call options dialog, rename the timer if desired.
5. Click OK. The timer instruction appears in the editor, and the IEC_TIMER structure appears in the FB Interface under Static.

6. If necessary, open the FB interface editor (may have to click on the small arrow to expand the view).
7. Under Static, locate the timer structure that was just created for you.
8. In the Retain column for this timer structure, change the selection to "Retain". Whenever this FB is called later from another program block, an instance DB will be created with this interface definition which contains the timer structure marked as retentive.

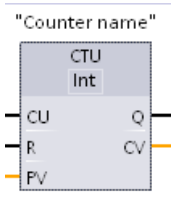
If the FB does not specify "Optimized block access", then the block access type is standard, which is compatible with S7-300/400 classic configurations and allows symbolic and direct access. To assign a multi-instance to a standard block access FB, follow these steps:

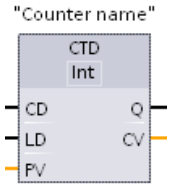
1. Open the FB for edit.
2. Place the timer instruction at the desired location in the FB.
3. When the Call options dialog appears, click on the multi instance icon. The multi instance option is only available if the instruction is being placed into an FB.
4. In the Call options dialog, rename the timer if desired.
5. Click OK. The timer instruction appears in the editor, and the IEC_TIMER structure appears in the FB Interface under Static.
6. Open the block that will use this FB.
7. Place this FB at the desired location. Doing so results in the creation of an instance data block for this FB.
8. Open the instance data block created when you placed the FB in the editor.
9. Under Static, locate the timer structure of interest. In the Retain column for this timer structure, check the box to make this structure retentive.

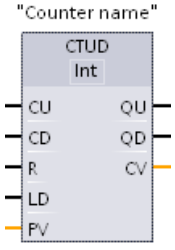
8.3 Counter operations

Table 8-23 Counter instructions

LAD / FBD	SCL	Description

	`"IEC_Counter_0_DB".CTU(`
	`CU:=_bool_in,`
	`R:=_bool_in,`
	`PV:=_in,`
	`Q=>_bool_out,`
	`CV=>_out);`
		Use the counter instructions to count internal program events and external process events. Each counter uses a structure stored in a data block to maintain counter data. You assign the data block when the counter instruction is placed in the editor.
		• CTU is a count-up counter
		• CTD is a count-down counter
		• CTUD is a count-up-and-down counter

	`"IEC_Counter_0_DB".CTD(`
	`CD:=_bool_in,`
	`LD:=_bool_in,`
	`PV:=_in,`
	`Q=>_bool_out,`
	`CV=>_out);`	

	`"IEC_Counter_0_DB".CTUD(`
	`CU:=_bool_in,`
	`CD:=_bool_in,`
	`R:=_bool_in,`
	`LD:=_bool_in,`
	`PV:=_in_,`
	`QU=>_bool_out,`
	`QD=>_bool_out,`
	`CV=>_out_);`	

[1] For LAD and FBD: Select the count value data type from the drop-down list below the instruction name.

[2] STEP 7 automatically creates the DB when you insert the instruction.

[3] In the SCL examples, "IEC_Counter_0_DB" is the name of the instance DB.

Table 8-24 Data types for the parameters

Parameter	Data type[1]	Description
CU, CD	Bool	Count up or count down, by one count
R (CTU, CTUD)	Bool	Reset count value to zero
LD (CTD, CTUD)	Bool	Load control for preset value
PV	SInt, Int, DInt, USInt, UInt, UDInt	Preset count value
Q, QU	Bool	True if CV >= PV
QD	Bool	True if CV <= 0
CV	SInt, Int, DInt, USInt, UInt, UDInt	Current count value

[1] The numerical range of count values depends on the data type you select. If the count value is an unsigned integer type, you can count down to zero or count up to the range limit. If the count value is a signed integer, you can count down to the negative integer limit and count up to the positive integer limit.

The number of counters that you can use in your user program is limited only by the amount of memory in the CPU. Counters use the following amount of memory:

- For SInt or USInt data types, the counter instruction uses 3 bytes.
- For Int or UInt data types, the counter instruction uses 6 bytes.
- For DInt or UDInt data types, the counter instruction uses 12 bytes.

These instructions use software counters whose maximum counting rate is limited by the execution rate of the OB in which they are placed. The OB that the instructions are placed in must be executed often enough to detect all transitions of the CU or CD inputs. For faster counting operations, see the CTRL_HSC instruction (Page 511).

Note

When you place counter instructions in an FB, you can select the multi-instance DB option, the counter structure names can be different with separate data structures, but the counter data is contained in a single DB and does not require a separate DB for each counter. This reduces the processing time and data storage necessary for the counters. There is no interaction between the counter data structures in the shared multi-instance DB.

Operation of the counters

Table 8-25 Operation of CTU (count up)

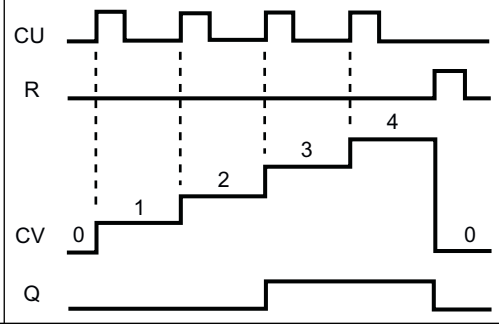
Counter	Operation
The CTU counter counts up by 1 when the value of parameter CU changes from 0 to 1. The CTU timing diagram shows the operation for an unsigned integer count value (where PV = 3). • If the value of parameter CV (current count value) is greater than or equal to the value of parameter PV (preset count value), then the counter output parameter Q = 1. • If the value of the reset parameter R changes from 0 to 1, then the current count value is reset to 0.	

Table 8-26 Operation of CTD (count down)

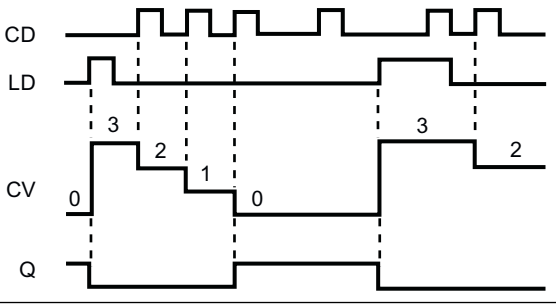
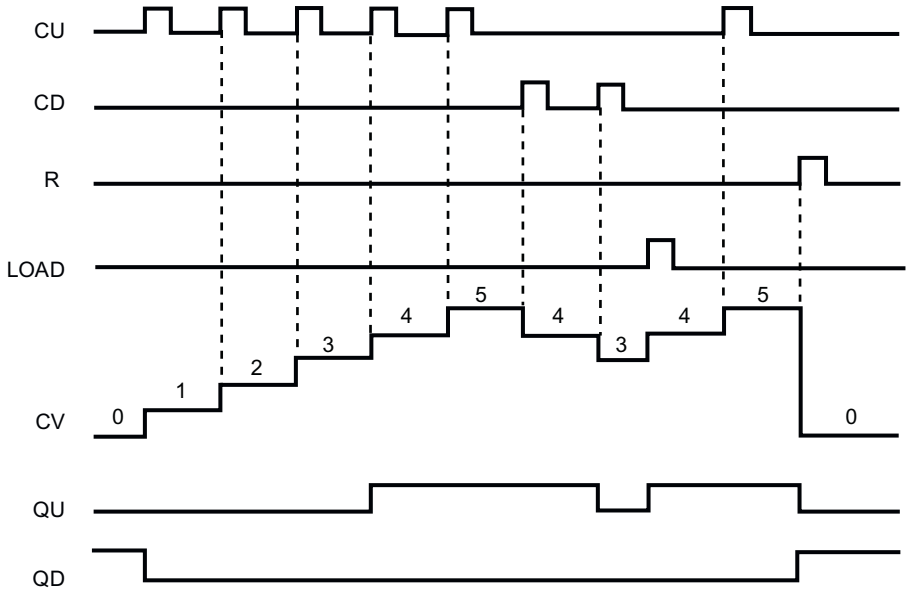
Counter	Operation
The CTD counter counts down by 1 when the value of parameter CD changes from 0 to 1. The CTD timing diagram shows the operation for an unsigned integer count value (where PV = 3). • If the value of parameter CV (current count value) is equal to or less than 0, the counter output parameter Q = 1. • If the value of parameter LOAD changes from 0 to 1, the value at parameter PV (preset value) is loaded to the counter as the new CV (current count value).	

Table 8-27 Operation of CTUD (count up and down)

Counter	Operation
The CTUD counter counts up or down by 1 on the 0 to 1 transition of the count up (CU) or count down (CD) inputs. The CTUD timing diagram shows the operation for an unsigned integer count value (where PV = 4). • If the value of parameter CV is equal to or greater than the value of parameter PV, then the counter output parameter QU = 1. • If the value of parameter CV is less than or equal to zero, then the counter output parameter QD = 1. • If the value of parameter LOAD changes from 0 to 1, then the value at parameter PV is loaded to the counter as the new CV. • If the value of the reset parameter R is changes from 0 to 1, the current count value is reset to 0.	

Counter data retention after a RUN-STOP-RUN transition or a CPU power cycle

If a run mode session is ended with stop mode or a CPU power cycle and a new run mode session is started, then the counter data stored in the previous run mode session is lost, unless the counter data structure is specified as retentive (CTU, CTD, and CTUD counters).

When you accept the defaults in the call options dialog after you place a counter instruction in the program editor, you are automatically assigned an instance DB which **cannot be made retentive**. To make your counter data retentive, you must either use a global DB or a Multi-instance DB.

Assign a global DB to store counter data as retentive data

This option works regardless of where the counter is placed (OB, FC, or FB).

1. Create a global DB:
   - Double-click "Add new block" from the Project tree
   - Click the data block (DB) icon
   - For the Type, choose global DB
   - If you want to be able to select individual items in this DB as retentive, be sure the symbolic-access-only box is checked.
   - Click OK
2. Add counter structure(s) to the DB:
   - In the new global DB, add a new static tag using one of the counter data types. Be sure to consider the Type you want to use for your Preset and Count values.
   - In the "Retain" column, check the box so that this structure will be retentive.
   - Repeat this process to create structures for all the counters that you want to store in this DB. You can either place each counter structure in a unique global DB, or you can place multiple counter structures into the same global DB. You can also place other static tags besides counters in this global DB. Placing multiple counter structures into the same global DB allows you to reduce your overall number of blocks.
   - Rename the counter structures if desired.
3. Open the program block for editing where you want to place a retentive counter (OB, FC, or FB).
4. Place the counter instruction at the desired location.
5. When the call options dialog appears, click the cancel button. You should now see a new counter instruction which has "???" both just above and just below the instruction name.
6. On the top of the new counter instruction, type the name (do not use the helper to browse) of the global DB and counter structure that you created above (example: "Data_block_3.Static_1"). This causes the corresponding preset and count value type to be filled in (example: UInt for an IEC_UCounter structure).

Counter Data Type	Corresponding Type for the Preset and Count Values
IEC_Counter	INT
IEC_SCounter	SINT
IEC_DCounter	DINT
IEC_UCounter	UINT
IEC_USCounter	USINT
IEC_UDCounter	UDINT

Assign a multi-instance DB to store counter data as retentive data

This option only works if you place the counter in an FB.

This option depends upon whether the FB properties specify "Optimized block access" (allows symbolic access only). To verify how the access attribute is configured for an existing FB, right-click on the FB in the Project tree, choose properties, and then choose Attributes.

If the FB specifies "Optimized block access" (allows symbolic access only):

1. Open the FB for edit.
2. Place the counter instruction at the desired location in the FB.
3. When the Call options dialog appears, click on the Multi instance icon. The Multi Instance option is only available if the instruction is being placed into an FB.
4. In the Call options dialog, rename the counter if desired.
5. Click OK. The counter instruction appears in the editor with type INT for the preset and count values, and the IEC_COUNTER structure appears in the FB Interface under Static.
6. If desired, change the type in the counter instruction from INT to one of the other types. The counter structure will change correspondingly.
7. If necessary, open the FB interface editor (may have to click on the small arrow to expand the view).
8. Under Static, locate the counter structure that was just created for you.
9. In the Retain column for this counter structure, change the selection to "Retain". Whenever this FB is called later from another program block, an instance DB will be created with this interface definition which contains the counter structure marked as retentive.

If the FB does not specify "Optimized block access", then the block access type is standard, which is compatible with S7-300/400 classic configurations and allows symbolic and direct access. To assign a multi-instance to a standard block access FB, follow these steps:

1. Open the FB for edit.
2. Place the counter instruction at the desired location in the FB.
3. When the Call options dialog appears, click on the multi instance icon. The multi instance option is only available if the instruction is being placed into an FB.
4. In the Call options dialog, rename the counter if desired.
5. Click OK. The counter instruction appears in the editor with type INT for the preset and count value, and the IEC_COUNTER structure appears in the FB Interface under Static.
6. If desired, change the type in the counter instruction from INT to one of the other types. The counter structure will change correspondingly.
7. Open the block that will use this FB.
8. Place this FB at the desired location. Doing so results in the creation of an instance data block for this FB.
9. Open the instance data block created when you placed the FB in the editor.
10. Under Static, locate the counter structure of interest. In the Retain column for this counter structure, check the box to make this structure retentive.

Type shown in counter instruction (for preset and count values)	Corresponding structure Type shown in FB interface
INT	IEC_Counter

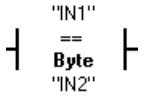
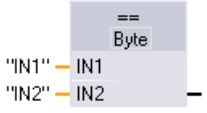
SINT	IEC_SCounter
DINT	IEC_DCounter
UINT	IEC_UCounter
USINT	IEC_USCounter
UDINT	IEC_UDCounter

8.4 Comparator operations

8.4.1 Compare values instructions

Table 8-28 Compare instructions

LAD	FBD	SCL	Description

		`out := in1 = in2;`
		or
		`IF in1 = in2`
		`  THEN out := 1;`
		`  ELSE out := 0;`
		`  END_IF;`
			Compares two values of the same data type. When the LAD contact comparison is TRUE, then the contact is activated. When the FBD box comparison is TRUE, then the box output is TRUE.

[1] For LAD and FBD: Click the instruction name (such as "==") to change the comparison type from the drop-down list. Click the "???" and select data type from the drop-down list.

Table 8-29 Data types for the parameters

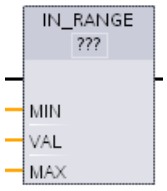
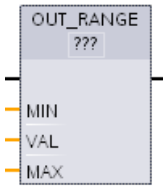
Parameter	Data type	Description
IN1, IN2	Byte, Word, DWord, SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, String, WString, Char, Char, Time, Date, TOD, DTL, Constant	Values to compare

Table 8-30 Comparison descriptions

Relation type	The comparison is true if ...
=	IN1 is equal to IN2
<>	IN1 is not equal to IN2
>=	IN1 is greater than or equal to IN2
<=	IN1 is less than or equal to IN2
>	IN1 is greater than IN2
<	IN1 is less than IN2

8.4.2 IN_Range (Value within range) and OUT_Range (Value outside range)

Table 8-31 Value within Range and value outside range instructions

LAD / FBD	SCL	Description
	`out := IN_RANGE(min, val, max);`	Tests whether an input value is in or out of a specified value range. If the comparison is TRUE, then the box output is TRUE.
	`out := OUT_RANGE(min, val, max);`	

[1] For LAD and FBD: Click the "???" and select the data type from the drop-down list.

Table 8-32 Data types for the parameters

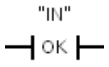
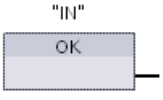
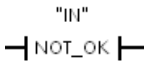
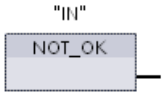
Parameter	Data type[1]	Description
MIN, VAL, MAX	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Constant	Comparator inputs

[1] The input parameters MIN, VAL, and MAX must be the same data type.

- The IN_RANGE comparison is true if: MIN <= VAL <= MAX
- The OUT_RANGE comparison is true if: VAL < MIN or VAL > MAX

8.4.3 OK (Check validity) and NOT_OK (Check invalidity)

Table 8-33 OK (check validity) and Not OK (check invalidity) instructions

LAD	FBD	SCL	Description
		Not available	Tests whether an input data reference is a valid real number according to IEEE specification 754.
		Not available	

[1] For LAD and FBD: When the LAD contact is TRUE, the contact is activated and passes power flow. When the FBD box is TRUE, then the box output is TRUE.

Table 8-34 Data types for the parameter

Parameter	Data type	Description
IN	Real, LReal	Input data

Table 8-35 Operation

Instruction	The Real number test is TRUE if:
OK	The input value is a valid real number [1]
NOT_OK	The input value is not a valid real number [1]

[1] A Real or LReal value is invalid if it is +/- INF (infinity), NaN (Not a Number), or if it is a denormalized value. A denormalized value is a number very close to zero. The CPU substitutes a zero for a denormalized value in calculations.

8.4.4 Variant and array comparison instructions

8.4.4.1 Equality and non-equality comparison instructions

The S7-1200 CPU provides instructions for querying the data type of a tag to which a Variant operand points for either equality or non-equality to the data type of the other operand.

In addition, the S7-1200 CPU provides instructions for querying the data type of an array element for either equality or non-equality to the data type of the other operand.

In these instructions, you are comparing <Operand1> to <Operand2>. <Operand1> must have the Variant data type. <Operand2> can be an elementary data type of a PLC data type. In LAD and FBD, <Operand1> is the operand above the instruction. In LAD, <Operand2> is the operand below the instruction.

For all instructions, the result of logic operation (RLO) is 1 (true) if the equality or non-equality test passes, and is 0 (false) if not.

The equality and non-equality type comparison instructions are as follows:

- EQ_Type (Compare data type for EQUAL with the data type of a tag)
- NE_Type (Compare data type for UNEQUAL with the data type of a tag)
- EQ_ElemType (Compare data type of an ARRAY element for EQUAL with the data type of a tag)
- NE_ElemType (Compare data type of an ARRAY element for UNEQUAL with the data type of a tag)

Table 8-36 EQ and NE instructions

LAD	FBD	SCL	Description
#Operand1 EQ_Type "Operand2"	#Operand1 EQ_Type "Operand2" — IN2 OUT	Not available	Tests whether the tag pointed to by the Variant at Operand1 is of the same data type as the tag at Operand2.
#Operand1 NE_Type "Operand2"	#Operand1 NE_Type "Operand2" — IN2 OUT	Not available	Tests whether the tag pointed to by the Variant at Operand1 is of a different data type as the tag at Operand2.

LAD	FBD	SCL	Description
#Operand1 EQ_ElemType "Operand2"	#Operand1 EQ_ElemType "Operand2" — IN2 OUT	Not available	Tests whether the array element pointed to by the Variant at Operand1 is of the same data type as the tag at Operand2.
#Operand1 NE_ElemType "Operand2"	#Operand1 NE_ElemType "Operand2" — IN2 OUT	Not available	Tests whether the array element pointed to by the Variant at Operand1 is of a different data type as the tag at Operand2.

Table 8-37 Data types for the parameters

Parameter	Data type	Description
Operand1	Variant	First operand
Operand2	Bit strings, integers, floating-point numbers, timers, date and time, character strings, ARRAY, PLC data types	Second operand

8.4.4.2 Null comparsion instructions

You can use the instructions IS_NULL and NOT_NULL to determine whether or not the input actually points to an object or not.

For both instructions, <Operand> must have the Variant data type.

Table 8-38 IS_NULL (Query for EQUALS ZERO pointer) and NOT_NULL (Query for EQUALS ZERO pointer) instructions

LAD	FBD	SCL	Description
#Operand IS_NULL	#Operand IS_NULL OUT	Not available	Tests whether the tag pointed to by the Variant at Operand is null and therefore not an object.
#Operand NOT_NULL	#Operand NOT_NULL OUT	Not available	Tests whether the tag pointed to by the Variant at Operand is not null and therefore does point to an object.

Table 8-39 Data types for the parameters

Parameter	Data type	Description
Operand	Variant	Operand to evaluate for null or not null.

8.4.4.3 IS_ARRAY (Check for ARRAY)

You can use the "Check for ARRAY" instruction to query whether the Variant points to a tag of the Array data type.

The <Operand> must have the Variant data type.

The instructions returns 1 (true) if the operand is an array.

Table 8-40 IS_ARRAY (Check for ARRAY)

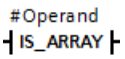
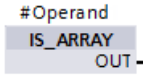
LAD	FBD	SCL	Description
		IS_ARRAY(_variant_in_)	Tests whether the tag pointed to by the Variant at Operand is an array.

Table 8-41 Data types for the parameters

Parameter	Data type	Description
Operand	Variant	Operand to evaluate for whether it is an array.

8.5 Math functions

8.5.1 CALCULATE (Calculate)

Table 8-42 CALCULATE instruction

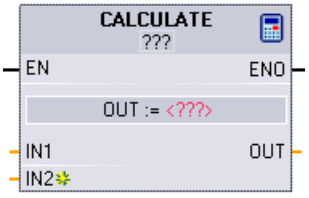
LAD / FBD	SCL	Description
	Use the standard SCL math expressions to create the equation.	The CALCULATE instruction lets you create a math function that operates on inputs (IN1, IN2, .. INn) and produces the result at OUT, according to the equation that you define. • Select a data type first. All inputs and the output must be the same data type. • To add another input, click the icon at the last input.

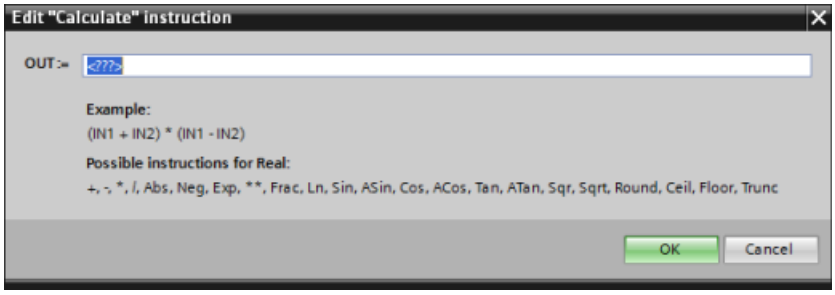
Table 8-43 Data types for the parameters

Parameter	Data type[1]
IN1, IN2, ..INn	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord

[1] The IN and OUT parameters must be the same data type (with implicit conversions of the input parameters). For example: A SINT value for an input would be converted to an INT or a REAL value if OUT is an INT or REAL

Click the calculator icon to open the dialog and define your math function. You enter your equation as inputs (such as IN1 and IN2) and operations. When you click "OK" to save the function, the dialog automatically creates the inputs for the CALCULATE instruction.

The dialog shows an example and a list of possible instructions that you can include based on the data type of the OUT parameter:



Note

You also must create an input for any constants in your function. The constant value would then be entered in the associated input for the CALCULATE instruction.

By entering constants as inputs, you can copy the CALCULATE instruction to other locations in your user program without having to change the function. You then can change the values or tags of the inputs for the instruction without modifying the function.

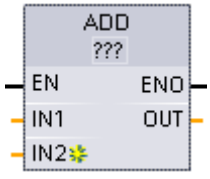
When CALCULATE is executed and all the individual operations in the calculation complete successfully, then the ENO = 1. Otherwise, ENO = 0.

For an example of the CALCULATE instruction, see "AUTOHOTSPOT".

8.5.2 Add, subtract, multiply and divide instructions

Table 8-44 Add, subtract, multiply and divide instructions

LAD / FBD	SCL	Description

	`out := in1 + in2;`
	`out := in1 - in2;`
	`out := in1 * in2;`
	`out := in1 / in2;`
		• ADD: Addition (IN1 + IN2 = OUT)
		• SUB: Subtraction (IN1 - IN2 = OUT)
		• MUL: Multiplication (IN1 * IN2 = OUT)
		• DIV: Division (IN1 / IN2 = OUT)
		An Integer division operation truncates the fractional part of the quotient to produce an integer output.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-45 Data types for the parameters (LAD and FBD)

Parameter	Data type[1]	Description
IN1, IN2	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Constant	Math operation inputs
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal	Math operation output

[1] Parameters IN1, IN2, and OUT must be the same data type.

To add an ADD or MUL input, click the "Create" icon or right-click on an input stub for one of the existing IN parameters and select the "Insert input" command.

To remove an input, right-click on an input stub for one of the existing IN parameters (when there are more than the original two inputs) and select the "Delete" command.

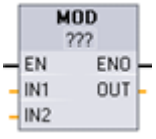
When enabled (EN = 1), the math instruction performs the specified operation on the input values (IN1 and IN2) and stores the result in the memory address specified by the output parameter (OUT). After the successful completion of the operation, the instruction sets ENO = 1.

Table 8-46 ENO status

ENO	Description
1	No error
0	The Math operation result value would be outside the valid number range of the data type selected. The least significant part of the result that fits in the destination size is returned.
0	Division by 0 (IN2 = 0): The result is undefined and zero is returned.
0	Real/LReal: If one of the input values is NaN (not a number) then NaN is returned.
0	ADD Real/LReal: If both IN values are INF with different signs, this is an illegal operation and NaN is returned.
0	SUB Real/LReal: If both IN values are INF with the same sign, this is an illegal operation and NaN is returned.
0	MUL Real/LReal: If one IN value is zero and the other is INF, this is an illegal operation and NaN is returned.
0	DIV Real/LReal: If both IN values are zero or INF, this is an illegal operation and NaN is returned.

8.5.3 MOD (return remainder of division)

Table 8-47 Modulo (return remainder of division) instruction

LAD / FBD	SCL	Description
	`out := in1 MOD in2;`	You can use the MOD instruction to return the remainder of an integer division operation. The value at the IN1 input is divided by the value at the IN2 input and the remainder is returned at the OUT output.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-48 Data types for parameters

Parameter	Data type[1]	Description
IN1 and IN2	SInt, Int, DInt, USInt, UInt, UDInt, Constant	Modulo inputs
OUT	SInt, Int, DInt, USInt, UInt, UDInt	Modulo output

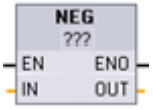
[1] The IN1, IN2, and OUT parameters must be the same data type.

Table 8-49 ENO values

ENO	Description
1	No error
0	Value IN2 = 0, OUT is assigned the value zero

8.5.4 NEG (Create twos complement)

Table 8-50 NEG (create twos complement) instruction

LAD / FBD	SCL	Description
	`-(in);`	The NEG instruction inverts the arithmetic sign of the value at parameter IN and stores the result in parameter OUT.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-51 Data types for parameters

Parameter	Data type[1]	Description
IN	SInt, Int, DInt, Real, LReal, Constant	Math operation input
OUT	SInt, Int, DInt, Real, LReal	Math operation output

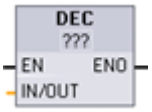
[1] The IN and OUT parameters must be the same data type.

Table 8-52 ENO status

ENO	Description
1	No error
0	The resulting value is outside the valid number range of the selected data type. Example for SInt: NEG (-128) results in +128 which exceeds the data type maximum.

8.5.5 INC (Increment) and DEC (Decrement)

Table 8-53 INC and DEC instructions

LAD / FBD	SCL	Description
	`in_out := in_out + 1;`	Increments a signed or unsigned integer number value: IN_OUT value +1 = IN_OUT value
	`in_out := in_out - 1;`	Decrements a signed or unsigned integer number value: IN_OUT value - 1 = IN_OUT value

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-54 Data types for parameters

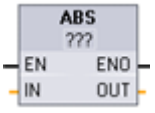
Parameter	Data type	Description
IN/OUT	SInt, Int, DInt, USInt, UInt, UDInt	Math operation input and output

Table 8-55 ENO status

ENO	Description
1	No error
0	The resulting value is outside the valid number range of the selected data type. Example for SInt: INC (+127) results in +128, which exceeds the data type maximum.

8.5.6 ABS (Form absolute value)

Table 8-56 ABS (absolute value) instruction

LAD / FBD	SCL	Description
	`out := ABS(in);`	Calculates the absolute value of a signed integer or real number at parameter IN and stores the result in parameter OUT.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-57 Data types for parameters

Parameter	Data type[1]	Description
IN	SInt, Int, DInt, Real, LReal	Math operation input
OUT	SInt, Int, DInt, Real, LReal	Math operation output

[1] The IN and OUT parameters must be the same data type.

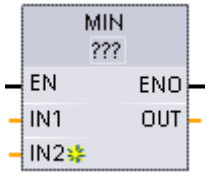
Table 8-58 ENO status

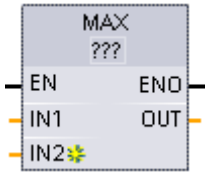
ENO	Description
1	No error
0	The math operation result value is outside the valid number range of the selected data type. Example for SInt: ABS (-128) results in +128 which exceeds the data type maximum.

8.5.7 MIN (Get minimum) and MAX (Get maximum)

Table 8-59 MIN (get minimum) and MAX (get maximum) instructions

LAD / FBD	SCL	Description

	`out:= MIN(`
	`  in1:=_variant_in_,`
	`  in2:=_variant_in_`
	`  [,...in32]);`
		The MIN instruction compares the value of two parameters IN1 and IN2 and assigns the minimum (lesser) value to parameter OUT.

	`out:= MAX(`
	`  in1:=_variant_in_,`
	`  in2:=_variant_in_`
	`  [,...in32]);`
		The MAX instruction compares the value of two parameters IN1 and IN2 and assigns the maximum (greater) value to parameter OUT.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-60 Data types for the parameters

Parameter	Data type[1]	Description
IN1, IN2 [...IN32]	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Time, Date, TOD, Constant	Math operation inputs (up to 32 inputs)
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Time, Date, TOD	Math operation output

[1] The IN1, IN2, and OUT parameters must be the same data type.

To add an input, click the "Create" icon or right-click on an input stub for one of the existing IN parameters and select the "Insert input" command.

To remove an input, right-click on an input stub for one of the existing IN parameters (when there are more than the original two inputs) and select the "Delete" command.

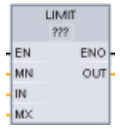
Table 8-61 ENO status

ENO	Description
1	No error
0	For Real data type only: • At least one input is not a real number (NaN). • The resulting OUT is +/- INF (infinity).

8.5.8 LIMIT (Set limit value)

Table 8-62 LIMIT (set limit value) instruction

LAD / FBD	SCL	Description

	`LIMIT(MN:=_variant_in_,`
	`IN:=_variant_in_,`
	`MX:=_variant_in_,`
	`OUT:=_variant_out_);`
		The Limit instruction tests if the value of parameter IN is inside the value range specified by parameters MIN and MAX and if not, clamps the value at MIN or MAX.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-63 Data types for the parameters

Parameter	Data type[1]	Description
MN, IN, and MX	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Time, Date, TOD· Constant	Math operation inputs
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Time, Date, TOD	Math operation output

[1] The MN, IN, MX, and OUT parameters must be the same data type.

If the value of parameter IN is within the specified range, then the value of IN is stored in parameter OUT. If the value of parameter IN is outside of the specified range, then the OUT value is the value of parameter MIN (if the IN value is less than the MIN value) or the value of parameter MAX (if the IN value is greater than the MAX value).

Table 8-64 ENO status

ENO	Description
1	No error
0	Real: If one or more of the values for MIN, IN and MAX is NaN (Not a Number), then NaN is returned.
0	If MIN is greater than MAX, the value IN is assigned to OUT.

SCL examples:

- MyVal := LIMIT(MN:=10,IN:=53, MX:=40); //Result: MyVal = 40
- MyVal := LIMIT(MN:=10,IN:=37, MX:=40); //Result: MyVal = 37
- MyVal := LIMIT(MN:=10,IN:=8, MX:=40); //Result: MyVal = 10

8.5.9 Exponent, logarithm, and trigonometry instructions

You use the floating point instructions to program mathematical operations using a Real or LReal data type:

- SQR: Form square (IN ^{2} = OUT)
- SQRT: Form square root (√IN = OUT)
- LN: Form natural logarithm (LN(IN) = OUT)
- EXP: Form exponential value (e ^{IN} =OUT), where base e = 2.71828182845904523536
- EXPT: exponentiate (IN1 ^{IN2} = OUT)
  EXPT parameters IN1 and OUT are always the same data type, for which you must select Real or LReal. You can select the data type for the exponent parameter IN2 from among many data types.
- FRAC: Return fraction (fractional part of floating point number IN = OUT)
- SIN: Form sine value (sin(IN radians) = OUT)
- ASIN: Form arcsine value (arcsine(IN) = OUT radians), where the sin(OUT radians) = IN
- COS: Form cosine (cos(IN radians) = OUT)
- ACOS: Form arccosine value (arccos(IN) = OUT radians), where the cos(OUT radians) = IN
- TAN: Form tangent value (tan(IN radians) = OUT)
- ATAN: Form arctangent value (arctan(IN) = OUT radians), where the tan(OUT radians) = IN

Table 8-65 Examples of floating-point math instructions

LAD / FBD	SCL	Description

	`out := SQR(in);`
	or
	`out := in * in;`
		Square: IN ^{2} = OUT
		For example: If IN = 9, then OUT = 81.

	`out := in1 ** in2;`	General exponential: IN1 ^{IN2} = OUT For example: If IN1 = 3 and IN2 = 2, then OUT = 9.

[1] For LAD and FBD: Click the "???" (by the instruction name) and select a data type from the drop-down menu.

[2] For SCL: You can also use the basic SCL math operators to create the mathematical expressions.

Table 8-66 Data types for parameters

Parameter	Data type	Description
IN, IN1	Real, LReal, Constant	Inputs
IN2	SInt, Int, DInt, USInt, UInt,UDInt, Real, LReal, Constant	EXPT exponent input
OUT	Real, LReal	Outputs

Table 8-67 ENO status

ENO	Instruction	Condition	Result (OUT)
1	All	No error	Valid result
0	SQR	Result exceeds valid Real/LReal range	+INF
		IN is +/- NaN (not a number)	+NaN
	SQRT	IN is negative	-NaN
		IN is +/- INF (infinity) or +/- NaN	+/- INF or +/- NaN
	LN	IN is 0.0, negative, -INF, or -NaN	-NaN
		IN is +INF or +NaN	+INF or +NaN
	EXP	Result exceeds valid Real/LReal range	+INF
		IN is +/- NaN	+/- NaN
	SIN, COS, TAN	IN is +/- INF or +/- NaN	+/- INF or +/- NaN
	ASIN, ACOS	IN is outside valid range of -1.0 to +1.0	+NaN
		IN is +/- NaN	+/- NaN
	ATAN	IN is +/- NaN	+/- NaN
	FRAC	IN is +/- INF or +/- NaN	+NaN
	EXPT	IN1 is +INF and IN2 is not -INF	+INF
		IN1 is negative or -INF	+NaN if IN2 is Real/LReal, -INF otherwise
		IN1 or IN2 is +/- NaN	+NaN
		IN1 is 0.0 and IN2 is Real/LReal (only)	+NaN

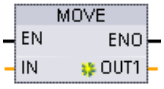
8.6 Move operations

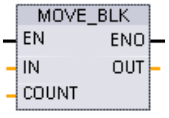
8.6.1 MOVE (Move value), MOVE_BLK (Move block), UMOVE_BLK (Move block uninterruptible), and MOVE_BLK_VARIANT (Move block)

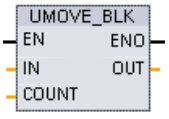
Use the Move instructions to copy data elements to a new memory address and convert from one data type to another. The source data is not changed by the move process.

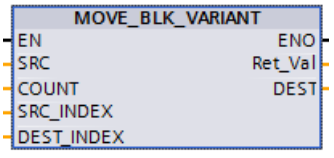
- The MOVE instruction copies a single data element from the source address specified by the IN parameter to the destination addresses specified by the OUT parameter.
- The MOVE_BLK and UMOVE_BLK instructions have an additional COUNT parameter. The COUNT specifies how many data elements are copied. The number of bytes per element copied depends on the data type assigned to the IN and OUT parameter tag names in the PLC tag table.

Table 8-68 MOVE, MOVE_BLK, UMOVE_BLK, and MOVE_BLK_VARIANT instructions

LAD / FBD	SCL	Description
	`out1 := in;`	Copies a data element stored at a specified address to a new address or multiple addresses.[1]

	`MOVE_BLK(`
	`in:=_variant_in,`
	`count:=_uint_in,`
	`out=>_variant_out);`
		Interruptible move that copies a block of data elements to a new address.

	`UMOVE_BLK(`
	`in:=_variant_in,`
	`count:=_uint_in,`
	`out=>_variant_out);`
		Uninterruptible move that copies a block of data elements to a new address.

	`MOVE_BLK(`
	`SRC:=_variant_in,`
	`COUNT:=_udint_in,`
	`SRC_INDEX:=_dint_in,`
	`DEST_INDEX:=_dint_in,`
	`DEST=>_variant_out);`
		Moves the contents of a source memory area to a destination memory area.
		You can copy a complete array or elements of an array to another array of the same data type. The size (number of elements) of source and destination array may be different. You can copy multiple or single elements within an array. You use Variant data types to point to both the source and destination arrays.

[1] MOVE instruction: To add another output in LAD or FBD, click the "Create" icon by the output parameter. For SCL, use multiple assignment statements. You might also use one of the loop constructions.

Table 8-69 Data types for the MOVE instruction

Parameter	Data type	Description
IN	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Char, WChar, Array, Struct, DTL, Time, Date, TOD, IEC data types, PLC data types	Source address
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Char, WChar, Array, Struct, DTL, Time, Date, TOD, IEC data types, PLC data types	Destination address

To add MOVE outputs, click the "Create" icon or right-click on an output stub for one of the existing OUT parameters and select the "Insert output" command.

To remove an output, right-click on an output stub for one of the existing OUT parameters (when there are more than the original two outputs) and select the "Delete" command.

Table 8-70 Data types for the MOVE_BLK and UMOVE_BLK instructions

Parameter	Data type	Description
IN	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal Byte, Word, DWord, Time, Date, TOD, WChar	Source start address
COUNT	UInt	Number of data elements to copy
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, WChar	Destination start address

Table 8-71 Data types for the MOVE_BLK_VARIANT instruction

Parameter	Data type	Description
SRC	Variant (which points to an array or individual array element)	Source block from which to copy
COUNT	UDInt	Number of data elements to copy
SRC_INDEX	DInt	Zero-based index into the SRC array
DEST_INDEX	DInt	Zero-based index into the DEST array
RET_VAL	Int	Error information
DEST	Variant (which points to an array or individualt array element)	Destination area into which to copy the contents of the source block

Note

Rules for data copy operations

- To copy the Bool data type, use SET_BF, RESET_BF, R, S, or output coil (LAD) (Page 200)
- To copy a single elementary data type, use MOVE
- To copy an array of an elementary data type, use MOVE_BLK or UMOVE_BLK
- To copy a structure, use MOVE
- To copy a string, use S_MOVE (Page 319)
- To copy a single character in a string, use MOVE
- The MOVE_BLK and UMOVE_BLK instructions cannot be used to copy arrays or structures to the I, Q, or M memory areas.

MOVE_BLK and UMOVE_BLK instructions differ in how interrupts are handled:

- Interrupt events are **queued and processed** during MOVE_BLK execution. Use the MOVE_BLK instruction when the data at the move destination address is not used within an interrupt OB subprogram or, if used, the destination data does not have to be consistent. If a MOVE_BLK operation is interrupted, then the last data element moved is complete and consistent at the destination address. The MOVE_BLK operation is resumed after the interrupt OB execution is complete.
- Interrupt events are **queued but not processed** until UMOVE_BLK execution is complete. Use the UMOVE_BLK instruction when the move operation must be completed and the destination data consistent, before the execution of an interrupt OB subprogram. For more information, see the section on data consistency (Page 177).

ENO is always true following execution of the MOVE instruction.

Table 8-72 ENO status

ENO	Condition	Result
1	No error	All COUNT elements were successfully copied.
0	Either the source (IN) range or the destination (OUT) range exceeds the available memory area.	Elements that fit are copied. No partial elements are copied.

Table 8-73 Condition codes for the MOVE_BLK_VARIANT instruction

RET_VAL (W#16#...)	Description
0000	No error
80B4	Data types do not correspond.
8151	Access to the SRC parameter is not possible.
8152	The operand at the SRC parameter is an invalid type.
8153	Code generation error at the SRC parameter
8154	The operand at the SRC parameter has the data type Bool.
8281	The COUNT parameter has an invalid value.
8382	The value at the SRC_INDEX parameter is outside the limits of the Variant.
8383	The value at parameter SRC_INDEX is outside the high limit of the array.
8482	The value at the DEST_INDEX parameter is outside the limits of the Variant.
8483	The value at parameter DEST_INDEX is outside the high limit of the array.
8534	The DEST parameter is write-protected.
8551	Access to the DEST parameter is not possible.
8552	The operand at the DEST parameter is an invalid type.
8553	Code generation error at the DEST parameter
8554	The operand at the DEST parameter has the data type Bool.
*You can display error codes in the program editor as integer or hexadecimal values.	

8.6.2 Deserialize

You can use the "Deserialize" instruction to convert the sequential representation of a PLC data type (UDT) back to a PLC data type and to fill its entire contents. If the comparison is TRUE, then the box output is TRUE.

The memory area which holds the sequential representation of a PLC data type must have the Array of Byte data type and you must declare the data block to have standard (not optimized) access. Make sure that there is enough memory space prior to the conversion.

The instruction enables you to convert multiple sequential representations of converted PLC data types back to their original data types.

Note

If you only want to convert back a single sequential representation of a PLC data type (UDT), you can also use the instruction "TRCV: Receive data via communication connection".

Table 8-74 DESERIALIZE instruction

LAD / FBD	SCL	Description

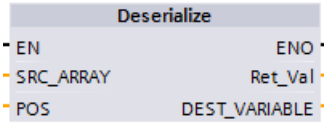
	`ret_val := Deserialize(`
	`  SRC_ARRAY:=_variant_in_,`
	`  DEST_VARIABLE=>_variant_out_`
	`,`
	`  POS:=_dint_inout_);`
		Converts the sequential representation of a PLC data type (UDT) back to a PLC data type and fills its entire contents

Table 8-75 Parameters for the DESERIALIZE instruction

Parameter	Type	Data type	Description
SRC_ARRAY	IN	Variant	Global data block that contains the data stream
DEST_VARIABLE	INOUT	Variant	Tag in which to store the converted PLC data type (UDT)
POS	INOUT	DInt	Number of bytes that the converted PLC data type uses
RET_VAL	OUT	Int	Error information

Table 8-76 RET_VAL parameter

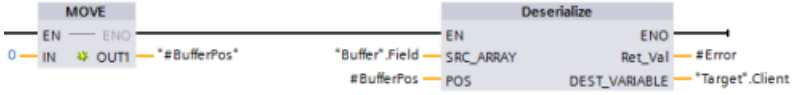
RET_VAL* (W#16#...)	Description
0000	No error
80B0	The memory areas for the SRC_ARRAY and DEST_VARIABLE parameters overlap.
8136	The data block at the DEST_VARIABLE parameter is not a block with standard access.
8150	The Variant data type at the SRC_ARRAY parameter contains no value.
8151	Code generation error at the SRC_ARRAY parameter.

RET_VAL* (W#16#...)	Description
8153	There is not enough free memory available at the SRC_ARRAY parameter.
8250	The Variant data type at the DEST_VARIABLE parameter contains no value.
8251	Code generation error at the DEST_VARIABLE parameter.
8254	Invalid data type at the DEST_VARIABLE parameter.
8382	The value at parameter POS is outside the limits of the array.
*You can view the error codes as either integer or hexadecimal in the program editor.	

Example: Deserialize instruction

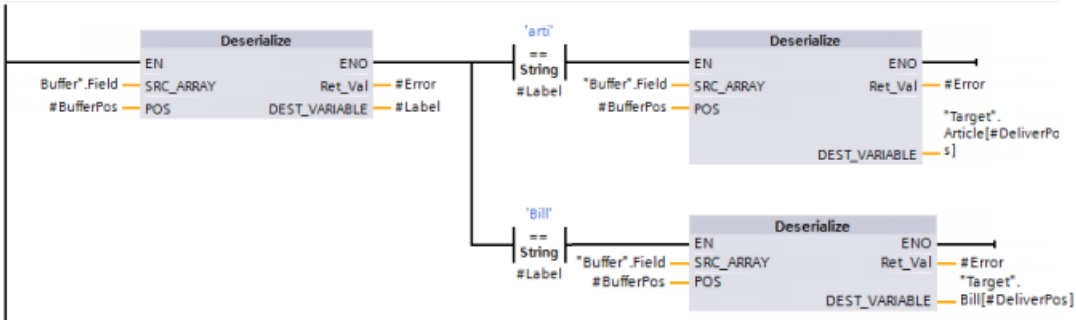
The following example shows how the instruction works:

Network 1:



The "MOVE" instruction moves the value "0" to the "#BufferPos" data block tag. The Deserialize instruction then deserializes the sequential representation of the customer data from the "Buffer" data block and writes it to the "Target" data block. The Deserialize instruction calculates the number of bytes that the converted data uses and stores it in the "#BufferPos" data block tag.

Network 2:



The "Deserialize" instruction deserializes the sequential representation of the data stream pointed to by "Buffer" and writes the characters to the "#Label" operand. The logic compares the characters using the comparison instructions "arti" and "Bill". If the comparison for "arti" = TRUE, the data is article data that is to be deserialized and written to the "Article" data structure of the "Target" data block. If the comparison for "Bill" = TRUE, the data is billing data that is to be deserialized and written to the "Bill" data structure of the "Target" data block.

Function block (or Function) interface:

	Name	Data type
	Input	
	DeliverPos	Int
	Output	
	InOut	
	Static	
	Temp	
	BufferPos	DInt
	Error	Int
	Label	String[4]

Custom PLC data types:

The structure of the two PLC data types (UDTs) for this example are as follows:

Article

	Name	Data type
1	Number	DInt
2	Declaration	String
3	Colli	Int

Client

	Name	Data type
1	Title	Int
2	Firstname	String[10]
3	Surname	String[10]

Data blocks:

The two data blocks for this example are as follows:

Target

	Name	Data type
1	Static	
2	Client	"Client"
3	Article	Array[0..10] of "Article"
4	Bill	Array[0..10] of Int

Buffer

	Name	Data type
1	Static	
2	Field	Array[0..294] of Byte

8.6.3 Serialize

You can use the "Serialize" instruction to convert several PLC data types (UDTs) to a sequential representation without any loss of structure.

You can use the instruction to temporarily save multiple structured data items from your program to a buffer, for example to a global data block, and send them to another CPU. The memory area in which the converted PLC data types are stored must have the ARRAY of BYTE data type and be declared with standard access. Make sure that there is enough memory space prior to the conversion.

The POS parameter contains information about the number of bytes that the converted PLC data types use.

Note

If you only want to send a single PLC data type (UDT), you can use the instruction "TSEND: Send data via communication connection".

Table 8-77 SERIALIZE instruction

LAD / FBD	SCL	Description

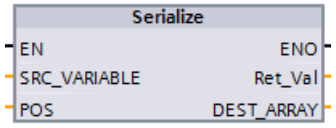
	`ret_val := Serialize(`
	`  SRC_VARIABLE=>_variant_in_,`
	`  DEST_ARRAY:=_variant_out_,`
	`  POS:=_dint_inout_);`
		Converts a PLC data type (UDT) to a sequential representation.

Table 8-78 Parameters for the SERIALIZE instruction

Parameter	Type	Data type	Description
SRC_VARIABLE	IN	Variant	PLC data type (UDT) that is to be converted to a serial representation
DEST_ARRAY	INOUT	Variant	Data block in which the generated data stream is to be stored
POS	INOUT	DInt	Number of bytes that the converted PLC data types use. The calculated POS parameter is zero-based.
RET_VAL	OUT	Int	Error information

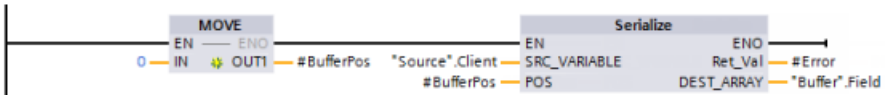
Table 8-79 RET_VAL parameter

RET_VAL* (W#16#...)	Description
0000	No error
80B0	The memory areas for the SRC_VARIABLE and DEST_ARRAY parameters overlap.
8150	The Variant data type at the SRC_VARIABLE parameter contains no value.
8152	Code generation error at the SRC_VARIABLE parameter.
8236	The data block at the DEST_ARRAY parameter is not a block with standard access.
8250	The Variant data type at the DEST_ARRAY parameter contains no value.
8252	Code generation error at the DEST_ARRAY parameter.
8253	There is not enough free memory available at the DEST_ARRAY parameter.
8254	Invalid data type at the DEST_VARIABLE parameter.
8382	The value at parameter POS is outside the limits of the array.
*You can view the error codes as either integer or hexadecimal in the program editor.		

Example: Serialize instruction

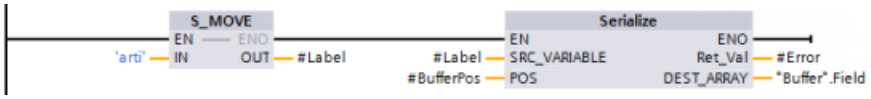
The following example shows how the instruction works:

Network 1:



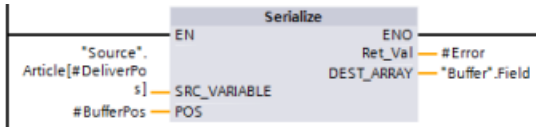
The "MOVE" instruction moves the value "0" to the "#BufferPos" parameter. The "Serialize" instruction serializes the customer data from the "Source" data block and writes it in sequential representation to the "Buffer" data block. The instruction stores the number of bytes used by the sequential representation in the "#BufferPos" parameter.

Network 2:



The logic now inserts some separator text to make it easier to deserialize the sequential representation later. The "S_MOVE" instruction moves the text string "arti" to the "#Label" parameter. The "Serialize" instruction writes these characters after the source client data to the "Buffer" data block. The instruction adds the number of bytes in the text string "arti" to the number already stored in the "#BufferPos" parameter.

Network 3:



The "Serialize" instruction serializes the data of a specific article, which is calculated in runtime, from the "Source" data block and writes it in sequential representation to the "Buffer" data block after the "arti" characters

Block Interface:

	Name	Data type
	Input	
	DeliverPos	Int
	Output	
	InOut	
	Static	
	Temp	
	BufferPos	DInt
	Error	Int
	Label	String[4]

Custom PLC data types:

The structure of the two PLC data types (UDTs) for this example are as follows:

Article

	Name	Data type
1	Number	DInt
2	Declaration	String
3	Colli	Int

Client

	Name	Data type
1	Title	Int
2	Firstname	String[10]
3	Surname	String[10]

Data blocks:

The two data blocks for this example are as follows:

Source

		Name	Data type
1		Static	
2		Client	"Client"
3		Article	Array[0..10] of "Article"

Buffer

		Name	Data type
1		Static	
2		Field	Array[0..294] of Byte

8.6.4 FILL_BLK (Fill block) and UFILL_BLK (Fill block uninterruptible)

Table 8-80 FILL_BLK and UFILL_BLK instructions

LAD / FBD	SCL	Description

	`FILL_BLK(`
	`  in:=_variant_in,`
	`  count:=int,`
	`  out=>_variant_out);`
		Interruptible fill instruction: Fills an address range with copies of a specified data element

	`UFILL_BLK(`
	`  in:=_variant_in,`
	`  count:=int,`
	`  out=>_variant_out);`
		Uninterruptible fill instruction: Fills an address range with copies of a specified data element

Table 8-81 Data types for parameters

Parameter	Data type	Description
IN	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Data source address
COUNT	UDint, USInt, UInt	Number of data elements to copy
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Data destination address

Note

Rules for data fill operations

- To fill with the BOOL data type, use SET_BF, RESET_BF, R, S, or output coil (LAD)
- To fill with a single elementary data type, use MOVE
- To fill an array with an elementary data type, use FILL_BLK or UFILL_BLK
- To fill a single character in a string, use MOVE
- The FILL_BLK and UFILL_BLK instructions cannot be used to fill arrays in the I, Q, or M memory areas.

The FILL_BLK and UFILL_BLK instructions copy the source data element IN to the destination where the initial address is specified by the parameter OUT. The copy process repeats and a block of adjacent addresses is filled until the number of copies is equal to the COUNT parameter.

FILL_BLK and UFILL_BLK instructions differ in how interrupts are handled:

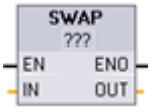
- Interrupt events are **queued and processed** during FILL_BLK execution. Use the FILL_BLK instruction when the data at the move destination address is not used within an interrupt OB subprogram or, if used, the destination data does not have to be consistent.
- Interrupt events are **queued but not processed** until UFILL_BLK execution is complete. Use the UFILL_BLK instruction when the move operation must be completed and the destination data consistent, before the execution of an interrupt OB subprogram.

Table 8-82 ENO status

ENO	Condition	Result
1	No error	The IN element was successfully copied to all COUNT destinations.
0	The destination (OUT) range exceeds the available memory area	Elements that fit are copied. No partial elements are copied.

8.6.5 SWAP (Swap bytes)

Table 8-83 SWAP instruction

LAD / FBD	SCL	Description
	`out := SWAP(in);`	Reverses the byte order for two-byte and four-byte data elements. No change is made to the bit order within each byte. ENO is always TRUE following execution of the SWAP instruction.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-84 Data types for the parameters

Parameter	Data type	Description
IN	Word, DWord	Ordered data bytes IN
OUT	Word, DWord	Reverse ordered data bytes OUT

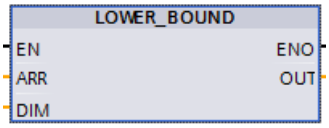
Example 1	Parameter IN = MB0 (before execution)		Parameter OUT = MB4, (after execution)	
Address	MW0	MB1	MW4	MB5
W#16#1234	12	34	34	12
WORD	MSB	LSB	MSB	LSB

Example 2	Parameter IN = MB0 (before execution)				Parameter OUT = MB4, (after execution)			
Address	MD0	MB1	MB2	MB3	MD4	MB5	MB6	MB7
DW#16# 12345678 DWORD	12 MSB	34	56	78 LSB	78 MSB	56	34	12 LSB

8.6.6 LOWER_BOUND: (Read out ARRAY low limit)

Table 8-85 LOWER_BOUND instruction

LAD / FBD	SCL	Description

	`out := LOWER_BOUND(`
	`ARR:=_variant_in_,`
	`DIM:=_udint_in_);`
		You can declare tags with ARRAY[*] in the block interface. For these local tags, you can read out the limits of the ARRAY. You will need to specify the required dimension at the DIM parameter.
		The LOWER_BOUND (Read out ARRAY low limit). instruction lets you read out the variable low limit of the ARRAY.

Parameters

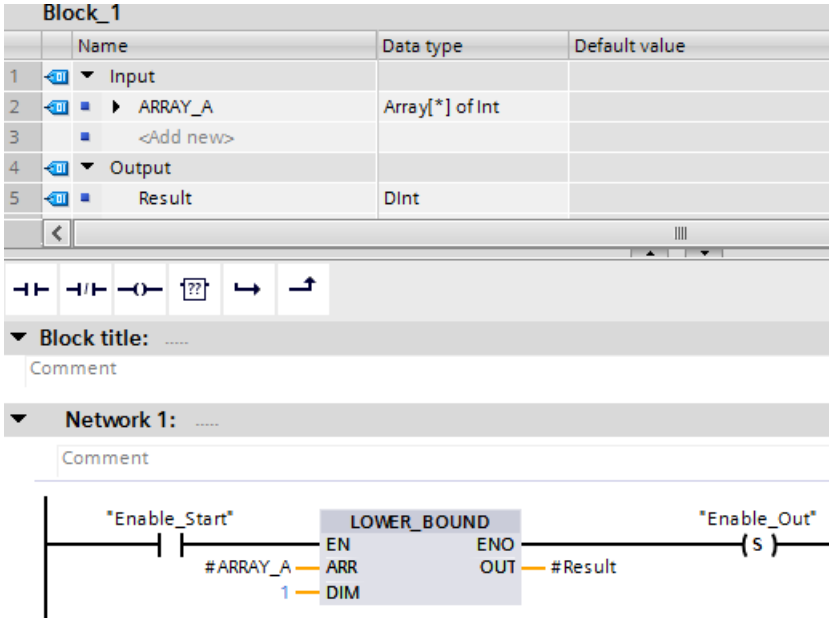
The following table shows the parameters of the instruction "LOWER_BOUND: Read out ARRAY low limit":

Parameters	Declaration	Data type	Memory area	Description
EN	Input	BOOL	I, Q, M, D, L	Enable input
ENO	Output	BOOL	I, Q, M, D, L	Enable output ENO has the signal state "0" if one of the following conditions applies: • The EN enable input has the signal state "0". • The dimension specified at input DIM does not exist.
ARR	Input	ARRAY [*]	FB: Section InOut FC: Sections Input and InOut	ARRAY of which the variable low limit is to be read.
DIM	Input	UDINT	I, Q, M, D, L or constant	Dimension of the ARRAY of which the variable low limit is to be read.
OUT	Output	DINT	I, Q, M, D, L	Result

You can find additional information on valid data types under "Data types (Page 100)":

Example

In the function (FC) block interface, the input parameter ARRAY_A is a one-dimensional array with variable dimensions.



If the "Enable_Start" operand returns signal state "1", the CPU executes the LOWER_BOUND instruction. It reads out the variable low limit of the ARRAY #ARRAY_A from the one-dimensional array. If the instruction executes without errors, it sets operand "Enable_Out" and sets the "Result" operand to the low limit of the array.

8.6.7 UPPER_BOUND: (Read out ARRAY high limit)

Table 8-86 LOWER_BOUND instruction

LAD / FBD	SCL	Description

UPPER_BOUND
EN ENO
ARR OUT
DIM
	`out := UPPER_BOUND(`
	`  ARR:=_variant_in_,`
	`  DIM:=_udint_in_);`
		You can declare tags with ARRAY[*] in the block interface. For these local tags, you can read out the limits of the ARRAY. You will need to specify the required dimension at the DIM parameter.
		The UPPER_BOUND (Read out ARRAY high limit) instruction lets you read out the variable high limit of the ARRAY.

Parameters

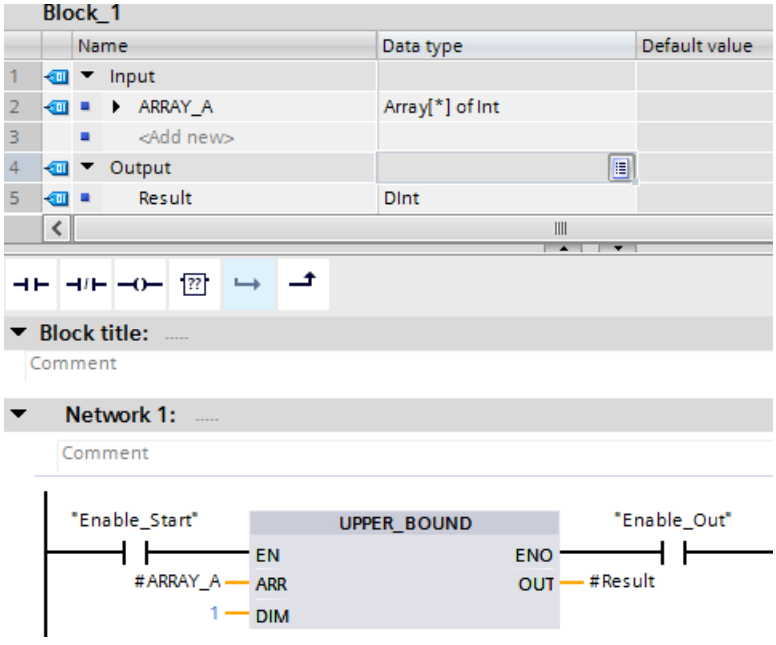
The following table shows the parameters of the instruction "UPPER_BOUND: Read out ARRAY high limit":

Parameters	Declaration	Data type	Memory area	Description
EN	Input	BOOL	I, Q, M, D, L	Enable input
ENO	Output	BOOL	I, Q, M, D, L	Enable output
ARR	Input	ARRAY [*]	FB: Section InOut FC: Sections Input and InOut	ARRAY of which the variable high limit is to be read.
DIM	Input	UDINT	I, Q, M, D, L or constant	Dimension of the ARRAY of which the variable high limit is to be read.
OUT	Output	DINT	I, Q, M, D, L	Result

You can find additional information on valid data types under "Data types (Page 100)":

Example

In the function (FC) block interface, the input parameter ARRAY_A is a one-dimensional array with variable dimensions.



If the "Enable_Start" operand returns signal state "1", the CPU executes the instruction. It reads out the variable high limit of the ARRAY #ARRAY_A from the one-dimensional array. If the instruction executes without errors, it sets operand "Enable_Out" and sets the "Result" operand.

8.6.8 Read / Write memory instructions

8.6.8.1 PEEK and POKE (SCL only)

SCL provides PEEK and POKE instructions that allow you to read from or write to data blocks, I/O, or memory. You provide parameters for specific byte offsets or bit offsets for the operation.

Note

To use the PEEK and POKE instructions with data blocks, you must use standard (not optimized) data blocks. Also note that the PEEK and POKE instructions merely transfer data. They have no knowledge of data types at the addresses.

```
PEEK(area:=_in_,
     dbNumber:=_in_,
     byteOffset:=_in_);
```

Reads the byte referenced by byteOffset of the referenced data block, I/O or memory area.

Example referencing data block:

```
%MB100 := PEEK(area:=16#84,
dbNumber:=1, byteOffset:=#i);
```

Example referencing IB3 input:

```
%MB100 := PEEK(area:=16#81,
dbNumber:=0, byteOffset:=#i); // when
#i = 3
```

```
PEEK_WORD(area:=_in_,
          dbNumber:=_in_,
          byteOffset:=_in_);
```

Reads the word referenced by byteOffset of the referenced data block, I/O or memory area.

Example:

```
%MW200 := PEEK_WORD(area:=16#84,
dbNumber:=1, byteOffset:=#i);
```

```
PEEK_DWORD(area:=_in_,
          dbNumber:=_in_,
          byteOffset:=_in_);
```

Reads the double word referenced by byteOffset of the referenced data block, I/O or memory area.

Example:

```
%MD300 := PEEK_DWORD(area:=16#84,
dbNumber:=1, byteOffset:=#i);
```

```
PEEK_BOOL(area:=_in_,
          dbNumber:=_in_,
          byteOffset:=_in_,
          bitOffset:=_in_);
```

Reads a Boolean referenced by the bitOffset and byteOffset of the referenced data block, I/O or memory area

Example:

```
%MB100.0 := PEEK_BOOL(area:=16#84,
dbNumber:=1, byteOffset:=#ii,
bitOffset:=#j);
```

```
POKE(area:=_in_,
     dbNumber:=_in_,
     byteOffset:=_in_,
     value:=_in_);
```

Writes the value (Byte, Word, or DWord) to the referenced byteOffset of the referenced data block, I/O or memory area

Example referencing data block:

```
POKE(area:=16#84, dbNumber:=2,
byteOffset:=3, value:="Tag_1");
```

Example referencing QB3 output:

```
POKE(area:=16#82, dbNumber:=0,
byteOffset:=3, value:="Tag_1");
```

```
POKE_BOOL(area:=_in_,
     dbNumber:=_in_,
     byteOffset:=_in_,
     bitOffset:=_in_,
     value:=_in_);
```

Writes the Boolean value to the referenced bitOffset and byteOffset of the referenced data block, I/O or memory area

Example:

```
POKE_BOOL(area:=16#84, dbNumber:=2,
byteOffset:=3, bitOffset:=5,
value:=0);
```

```
POKE_BLK(area_src:=_in_,
     dbNumber_src:=_in_,
     byteOffset_src:=_in_,
     area_dest:=_in_,
     dbNumber_dest:=_in_,
     byteOffset_dest:=_in_,
     count:=_in_);
```

Writes "count" number of bytes starting at the referenced byte Offset of the referenced source data block, I/O or memory area to the referenced byteOffset of the referenced destination data block, I/O or memory area

Example:

```
POKE_BLK(area_src:=16#84,
dbNumber_src:=#src_db,
byteOffset_src:=#src_byte,
area_dest:=16#84,
dbNumber_dest:=#src_db,
byteOffset_dest:=#src_byte,
count:=10);
```

For PEEK and POKE instructions, the following values for the "area", "area_src" and "area_dest" parameters are applicable. For areas other than data blocks, the dbNumber parameter must be 0.

16#81	I
16#82	Q
16#83	M
16#84	DB

8.6.8.2 Read and write big and little Endian instructions (SCL)

The S7-1200 CPU provides SCL instructions for reading and writing data in little endian format and in big endian format. Little endian format means that the byte with the least significant bit is in the lowest memory address. Big endian format means that the byte with the most significant bit is in the lowest memory address.

The four SCL instructions for reading and writing data in little endian and big endian format are as follows:

- READ_LITTLE (Read data in little endian format)
- WRITE_LITTLE (Write data in little endian format)

- READ_BIG (Read data in big endian format)
- WRITE_BIG (Write data in big endian format)

Table 8-87 Read and write big and little endian instructions

LAD / FBD	SCL	Description

Not available	`READ_LITTLE(`
	`  src_array:=_variant_in_,`
	`  dest_Variable =>_out_,`
	`  pos:=_dint_inout)`
		Reads data from a memory area and writes it to a single tag in little endian byte format.

Not available	`WRITE_LITTLE(`
	`  src_variable:=_in_,`
	`  dest_array =>_variant_inout_,`
	`  pos:=_dint_inout)`
		Writes data from a single tag to a memory area in little endian byte format.

Not available	`READ_BIG(`
	`  src_array:=_variant_in_,`
	`  dest_Variable =>_out_,`
	`  pos:=_dint_inout)`
		Reads data from a memory area and writes it to a single tag in big endian byte format.

Not available	`WRITE_BIG(`
	`  src_variable:=_in_,`
	`  dest_array =>_variant_inout_,`
	`  pos:=_dint_inout)`
		Writes data from a single tag to a memory area in big endian byte format.

Table 8-88 Parameters for the READ_LITTLE and READ_BIG instructions

Parameter	Data type	Description
src_array	Array of Byte	Memory area from which to read data
dest_Variable	Bit strings, integers, floating-point numbers, timers, date and time, character strings	Destination variable at which to write data
pos	DINT	Zero-based position from which to start reading data from the src_array input.

Table 8-89 Parameters for the WRITE_LITTLE and WRITE_BIG instructions

Parameter	Data type	Description
src_variable	Bit strings, integers, floating-point numbers, TOD, DATA, Char, WChar	Source data from tag
dest_array	Array of Byte	Memory area at which to write data
pos	DINT	Zero-based position at which to start writing data into the dest_array output.

Table 8-90 RET_VAL parameter

RET_VAL* (W#16#...)	Description
0000	No error
80B4	The SRC_ARRAY or DEST_ARRAY is not an Array of Byte
8382	The value at parameter POS is outside the limits of the array.
8383	The value at parameter POS is within the limits of the Array but the size of the memory area exceeds the high limit of the array.
*You can view the error codes as either integer or hexadecimal in the program editor.	

8.6.9 Variant instructions

8.6.9.1 VariantGet (Read VARIANT tag value)

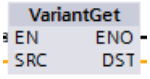
You can use the "Read out Variant tag value" instruction to read the value of the tag to which the Variant pointer at the SRC parameter points and write it in the tag at the DST parameter.

The SRC parameter has the Variant data type. Any data type except for Variant can be specified at the DST parameter.

The data type of the tag at the DST parameter must match the data type to which the Variant points.

Table 8-91 VariantGet instruction

LAD / FBD	SCL	Description

	`VariantGet(`
	`  SRC:=_variant_in_,`
	`  DST=>_variant_out_);`
		Reads the tag pointed to by the SRC parameter and writes it to the tag at the DST parameter

Note

To copy structures and arrays, you can use the "MOVE_BLK_VARIANT: Move block" instruction.

Table 8-92 Parameters for the VariantGet instruction

Parameter	Data type	Description
SRC	Variant	Pointer to source data
DST	Bit strings, integers, floating-point numbers, timers, date and time, character strings, ARRAY elements, PLC data types	Destination at which to write data

Table 8-93 ENO status

ENO	Condition	Result
1	No error	Instruction copied the tag data pointed to by SRC to the DST tag.
0	Enable input EN has the signal state "0" or the data types do not correspond.	Instruction copied no data.

8.6.9.2 VariantPut (Write VARIANT tag value)

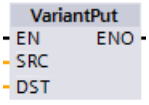
You can use the "Write VARIANT tag value" instruction to write the value of the tag at the SRC parameter to the tag at the DST parameter to which the VARIANT points.

The DST parameter has the VARIANT data type. Any data type except for VARIANT can be specified at the SRC parameter.

The data type of the tag at the SRC parameter must match the data type to which the VARIANT points.

Table 8-94 VariantPut instruction

LAD / FBD	SCL	Description

	`VariantPut(`
	`  SRC:=_variant_in_,`
	`  DST=>_variant_in_);`
		Writes the tag referenced by the SRC parameter to the variant pointed to by the DST parameter

Note

To copy structures and ARRAYs, you can use the "MOVE_BLK_VARIANT: Move block" instruction.

Table 8-95 Parameters for the VariantPut instruction

Parameter	Data type	Description
SRC	Bit strings, integers, floating-point numbers, timers, date and time, character strings, ARRAY elements, PLC data types	Pointer to source data
DST	Variant	Destination at which to write data

Table 8-96 ENO status

ENO	Condition	Result
1	No error	Instruction copied the SRC tag data to the DST tag.
0	Enable input EN has the signal state "0" or the data types do not correspond.	Instruction copied no data.

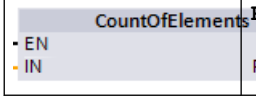
8.6.9.3 CountOfElements (Get number of ARRAY elements)

You can use the "Get number of ARRAY elements" instruction to query how many Array elements are in a tag pointed to by a Variant.

If it is a one-dimensional ARRAY, the instruction returns the difference between the high and low limit +1 is output. If it is a multi-dimensional ARRAY, the instruction returns the product of all dimensions.

Table 8-97 CountOfElements instruction

LAD / FBD	SCL	Description

	`Result := CountOfElements(`
	`ENO variant_in_);`
	`RET_VAL`
		Counts the number of array elements at the array pointed to by the IN parameter.

Note

If the Variant points to an Array of Bool, the instruction counts the fill elements to the nearest byte boundary. For example, the instruction returns 8 as the count for an Array[0..1] of Bool.

Table 8-98 Parameters for the CountOfElements instruction

Parameter	Data type	Description
IN	Variant	Tag with array elements to be counted
RET_VAL	UDint	Instruction result

Table 8-99 ENO status

ENO	Condition	Result
1	No error	Instruction returns the number of array elements.
0	Enable input EN has the signal state "0" or the Variant does not point to an array.	Instruction returns 0.

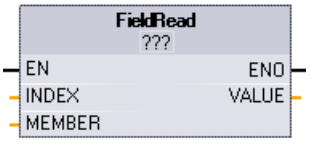
8.6.10 Legacy instructions

8.6.10.1 FieldRead (Read field) and FieldWrite (Write field) instructions

Note

STEP 7 V10.5 **did not support** a variable reference as an array index or multi-dimensional arrays. The FieldRead and FieldWrite instructions were used to provide variable array index operations for a one-dimensional array. STEP 7 V11 and greater **do support** a variable as an array index and multi-dimensional arrays. FieldRead and FieldWrite are included in STEP 7 V11 and greater for backward compatibility with programs that have used these instructions.

Table 8-100 FieldRead and FieldWrite instructions

LAD / FBD	SCL	Description
	`value := member[index];`	FieldRead reads the array element with the index value INDEX from the array whose first element in specified by the MEMBER parameter. The value of the array element is transferred to the location specified at the VALUE parameter.
	`member[index] := value;`	WriteField transfers the value at the location specified by the VALUE parameter to the array whose first element is specified by the MEMBER parameter. The value is transferred to the array element whose array index is specified by the INDEX parameter.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-101 Data types for parameters

Parameter and type		Data type	Description
Index	Input	DInt	The index number of the array element to be read or written to
Member [1]	Input	Binary numbers, integers, floating-point numbers, timers, DATE, TOD, CHAR and WCHAR as components of an ARRAY tag	Location of the first element in a one- dimension array defined in a global data block or block interface. For example: If the array index is specified as [-2..4], then the index of the first element is -2 and not 0.
Value [1]	Out	Binary numbers, integers, floating-point numbers, timers, DATE, TOD, CHAR, WCHAR	Location to which the specified array element is copied (FieldRead) Location of the value that is copied to the specified array element (FieldWrite)

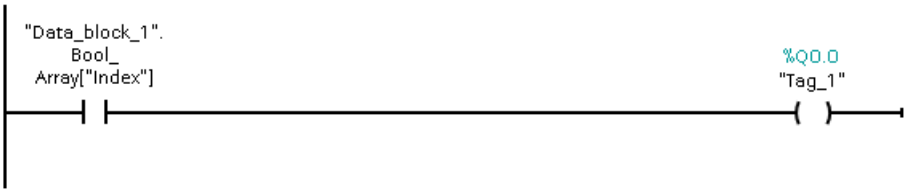
[1] The data type of the array element specified by the MEMBER parameter and the VALUE parameter must have the same data type.

The enable output ENO = 0, if one of the following conditions applies:

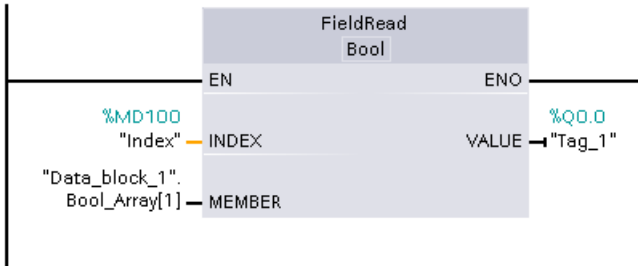
- The EN input has signal state "0"
- The array element specified at the INDEX parameter is not defined in the array referenced at MEMBER parameter
- Errors such as an overflow occur during processing

Example: Accessing data by array indexing

To access elements of an array with a variable, simply use the variable as an array index in your program logic. For example, the network below sets an output based on the Boolean value of an array of Booleans in "Data_block_1" referenced by the PLC tag "Index".



The logic with the variable array index is equivalent to the former method using the FieldRead instruction:



FieldWrite and FieldRead instructions can be replaced with variable array indexing logic.

SCL has no FieldRead or FieldWrite instructions, but supports indirect addressing of an array with a variable:

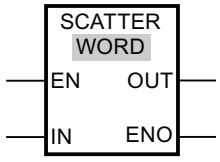
```
#Tag_1 := "Data_block_1".Bool_Array[#Index];
```

8.6.11 SCATTER

SCATTER: Parse the bit sequence into individual bits

The Parse the bit sequence into individual bits instruction parses a tag of the BYTE, WORD, or DWORD data type into individual bits and saves them in an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements.

Table 8-102 SCATTER

LAD/FBD	SCL	Description
	`SCATTER (IN := #SourceWord,` `OUT => #DestinationArray);`	The SCATTER: Parse the bit sequence into individual bits instruction parses a tag of the BYTE, WORD, or DWORD data type into individual bits and saves them in an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements.

Note

Multi-dimensional ARRAY of BOOL

With the "Parse the bit sequence into individual bits" instruction, the use of a multidimensional ARRAY of BOOL is not permitted.

Note

Length of the ARRAY, STRUCT or PLC data type

The ARRAY, the anonymous STRUCT or the PLC data type must have exactly the number of elements that is specified by the bit sequence. This means for the data type BYTE, for example, the ARRAY, STRUCT or the PLC data type must have exactly 8 elements (WORD = 16, and DWORD = 32).

Note

Note Availability of the instruction

The instruction can be used with a CPU of the S7-1200 series as of firmware version >4.2, and for a CPU of the S7-1500 series as of firmware version 2.1.

This way you can, for example, parse a status word, and read and change the status of the individual bits using the index. Using GATHER, you can merge the bits once again into a bit sequence.

The enable output ENO returns signal state "0" if one of the following conditions applies:

- Enable input EN has signal state "0".
- The ARRAY, STRUCT or PLC data type does not provide enough BOOL elements.

Data types for the SCATTER instruction

The following table shows the parameters of the instruction:

Parameter	Declaration	Data type		Memory area	Description
		S7-1200	S7-1500		
EN	Input	BOOL	BOOL	I, Q, M, D, L or constant	Enable input
ENO	Output	BOOL	BOOL	I, Q, M, D, L	Enable output
IN	Input	BYTE, WORD, DWORD	BYTE, WORD, DWORD	I, Q, M, D, L	Bit sequence that is parsed. The values must not be located in the I/O area or in the DB of a technology object.
OUT	Output	ARRAY[*] of BOOL, STRUCT or PLC data type *: 8, 16, 32 or 64 elements	ARRAY[*] of BOOL, STRUCT or PLC data type *: 8, 16, 32 or 64 elements	I, Q, M, D, L	ARRAY, STRUCT or PLC data type in which the individual bits are stored

You can find additional information on valid data types under "See also".

Example with an ARRAY

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceWord		WORD
EnableOut	Output	BOOL
DestinationArray		ARRAY[0..15] of BOOL

The following example shows how the instruction works:

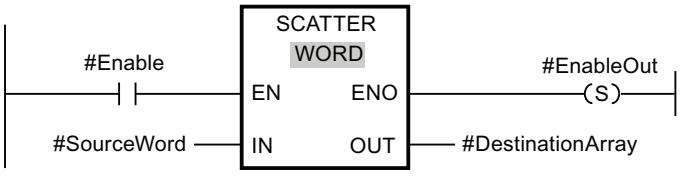


Figure 8-1 SCATTER example 2

The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceWord	WORD (16 bits)
OUT	DestinationUDT	The "DestinationUDT" operand has the PLC data type (UDT). It consists of 16 elements and is therefore just as large as the WORD that is to be parsed.

If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. The #SourceWord operand of the data type WORD is parsed into its individual bits (16) and assigned to the individual elements of the #DestinationArray operand. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

You can find additional information and the program code for the above-named example here: Sample Library for Instructions.

Example with a PLC data type (UDT)

Create the following PLC data type "myBits":

myBits

	Name	Data type
1	GeneralError	Bool
2	DeviceError	Bool
3	CommError	Bool
4	myError1	Bool
5	myError2	Bool
6	myError3	Bool
7	myError4	Bool
8	myError5	Bool
9	myError6	Bool
10	myError7	Bool
11	myError8	Bool
12	myError9	Bool
13	myError10	Bool
14	myError11	Bool
15	myError12	Bool
16	myError13	Bool

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceWord		WORD
EnableOut	Output	BOOL
DestinationUDT		"myBits"

The following example shows how the instruction works:

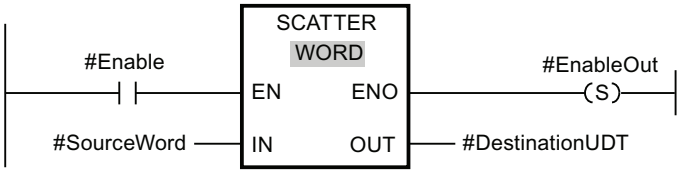


Figure 8-2 SCATTER_Bsp_example

The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceWord	WORD (16 bits)
OUT	DestinationUDT	The "DestinationUDT" operand has the PLC data type (UDT). It consists of 16 elements and is therefore just as large as the WORD that is to be parsed.

If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. The #SourceWord operand of the data type WORD is parsed into its individual bits (16) and assigned to the individual elements of the #DestinationArray operand. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

See also

New features (Page 35)

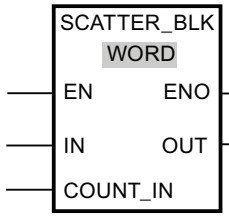
8.6.12 SCATTER_BLK

SCATTER_BLK: Parse elements of an ARRAY of bit sequence into individual bits

The "Parse elements of an ARRAY of bit sequence into individual bits" instruction parses one or more elements of an ARRAY of BYTE, WORD, or DWORD into individual bits and saves them in an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements. At the COUNT_IN parameter you specify how many elements of the source ARRAY are going to be parsed. The source ARRAY at the IN parameter may have more elements than specified at the COUNT_IN parameter. The ARRAY of BOOL, the anonymous STRUCT or the PLC data type must have sufficient elements to save the bits of the parsed bit sequences. However, the destination memory area may also be larger.

Table 8-103 SCATTER_BLK

LAD/FBD	SCL	Description

	`SCATTER_BLK(IN:=_byte_in_,`
	`COUNT_IN:=_uint_in_,`
	`OUT=>_bool_out_);_IN:=_uint_in_,`
		The "Parse elements of an ARRAY of bit sequence into individual bits" instruction parses one or more elements of an ARRAY of BYTE, WORD, or DWORD into individual bits and saves them in an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements. At the COUNT_IN parameter you specify how many elements of the source ARRAY are going to be parsed. The source ARRAY at the IN parameter may have more elements than specified at the COUNT_IN parameter. The ARRAY of BOOL, the anonymous STRUCT or the PLC data type must have sufficient elements to save the bits of the parsed bit sequences. However, the destination memory area may also be larger.

Note

NO data is written when ENO is false

In the S7-1200 CPU only for the SCATTER_BLK instruction, If ENO is FALSE, no data is written to the output.

Note

Multi-dimensional ARRAY of BOOL

If the ARRAY is a multi-dimensional ARRAY of BOOL, the filling bits of the dimensions contained are counted as well even if they were not explicitly declared.

Example 1: An ARRAY[1..10,0..4,1..2] of BOOL is handled like an ARRAY[1..10,0..4,1..8] of BOOL or like an ARRAY[0..399] of BOOL.

Example 2: At the IN parameter, an ARRAY[0..5] of WORD (sourceArrayWord[2]) is interconnected. The COUNT_IN parameter has the value "3". At the OUT parameter, an ARRAY[0..1,0..5,0..7] of BOOL (destinationArrayBool[0,0,0]) is interconnected. Both the array at the IN parameter and at the OUT parameter has a size of 96 bits. The ARRAY of WORD is parsed into 48 individual bits

Note

If the ARRAY low limit of the target ARRAY is not "0", note the following:

For performance reasons the index must always start at a BYTE, WORD, or DWORD limit. This means the index must be calculated starting at the low limit of the ARRAY. The following formula is used as basis for this calculation:

Valid indices = ARRAY low limit + n(number of bit sequences) × number of bits of the desired bit sequence

For an ARRAY[-2..45] of BOOL and the bit sequence WORD the calculation looks as follows:

- Valid index (-2) = -2 + 0 × 16
- Valid index (14) = -2 + 1 × 16
- Valid index (30) = -2 + 2 × 16

You can find an example described below.

Note

Availability of the instruction

The instruction can be used with a CPU of the S7-1200 series as of firmware version >4.2, and for a CPU of the S7-1500 series as of firmware version 2.1.

This way you can, for example, parse status words, and read and change the status of the individual bits using the index. Using GATHER, you can merge the bits once again into a bit sequence.

The enable output ENO returns signal state "0" if one of the following conditions applies:

- Enable input EN has signal state "0".
- The source ARRAY has fewer elements than specified at the COUNT_IN parameter.
- The index of the destination ARRAY does not start at a BYTE, WORD, or DWORD limit. In this case, no result is written to the ARRAY of BOOL.
- The ARRAY[*] of BOOL, STRUCT or PLC data type does not provide the required number of elements.
  - S7-1500-CPU: In this case as many bit sequences as possible are parsed and written to the ARRAY of BOOL, the anonymous STRUCT or the PLC data type. The remaining bit sequences are no longer taken into account.
  - S7-1200-CPU: There is no copying procedure.

Data types for the SCATTER_BLK instruction

The following table shows the parameters of the instruction:

Parameter	Declaration	Data type		Memory area	Description
		S7-1200	S7-1500		
EN	Input	BOOL	BOOL	I, Q, M, D, L or constant	Enable input
ENO	Output	BOOL	BOOL	I, Q, M, D, L	Enable output
IN	Input	Element of an ARRAY[*] of <bit sequence>	Element of an ARRAY[*] of <bit sequence>	I, Q, M, D, L	ARRAY of <bit sequence> that is parsed. The values must not be located in the I/O area or in the DB of a technology object.
COUNT_IN	Input	USINT, UINT, UDINT	USINT, UINT, UDINT	I, Q, M, D, L	Counter for the number of elements of the source ARRAY that are going to be parsed. The value must not be in the I/O area or in the database of a technology object.
OUT	Output	Element of an ARRAY[*] of BOOL, STRUCT or PLC data type	Element of an ARRAY[*] of BOOL, STRUCT or PLC data type	I, Q, M, D, L	ARRAY, STRUCT or PLC data type in which the individual bits are stored

You can select the required bit sequence from the "???" drop-down list of the instruction box.

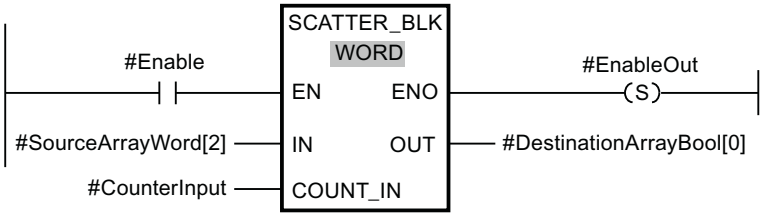
You can find additional information on valid data types under "See also".

Example of a destination ARRAY with the low limit "0"

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceArrayWord		ARRAY[0..5] of WORD
CounterInput		UDINT
EnableOut	Output	BOOL
DestinationArrayBool		ARRAY[0..95] of BOOL

The following example shows how the instruction works:



The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceArrayWord[2]	ARRAY[0..5] of WORD (96 bits can be parsed.)
COUNT_IN	CounterInput = 3	UDINT3 (3 WORDs or 48 bits are to be parsed. This means at least 48 bits must be available in the destination ARRAY.)
OUT	DestinationArrayBool[0]	The operand "DestinationArray-Bool" is of the data type AR-RAY[0..95] of BOOL. This means it provides 96 BOOL elements.

If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. The 3rd, 4th and 5th WORD of the #SourceArrayWord operand is parsed into its individual bits (48) and as of the 1st element assigned to the individual elements of the #DestinationArrayBool operand. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

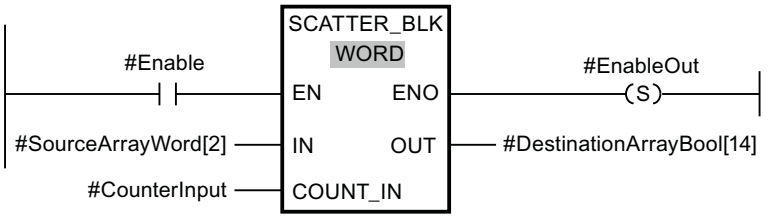
Example of a destination ARRAY with the low limit "-2"

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceArrayWord		ARRAY[0..5] of WORD
CounterInput		UDINT

Tag	Section	Data type
EnableOut	Output	BOOL
DestinationArrayBool		ARRAY[-2..93] of BOOL

The following example shows how the instruction works:



The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceArrayWord[2]	ARRAY[0..5] of WORD (96 bits can be parsed.)
COUNT_IN	CounterInput = 3	UDINT3 (3 WORDs or 48 bits are to be parsed. This means at least 48 bits must be available in the destination ARRAY.)
OUT	DestinationArrayBool[14]	The operand "DestinationArray-Bool" is of the data type AR-RAY[-2..93] of BOOL. This means it provides 96 BOOL elements.

If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. The 3rd, 4th and 5th WORD of the #SourceArrayWord operand is parsed into its individual bits (48) and as of the 16th Element assigned to the individual elements of the #DestinationArrayBool operand. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO. The remaining 32 bits are not written.

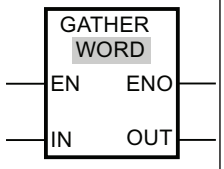
You can find additional information and the program code for the above-named example here: Sample Library for Instructions.

8.6.13 GATHER

GATHER

The "Merge individual bits into a bit sequence" instruction merges the bits from an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements into a bit sequence. The bit sequence is saved in a tag of the data type BYTE, WORD, DWORD or LWORD.

Table 8-104 GATHER

LAD/FBD	SCL	Description
	`GATHER(IN := #SourceArray, OUT => #DestinationArray};`	The GATHER: The "Merge individual bits into a bit sequence" instruction merges the bits from an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements into a bit sequence. The bit sequence is saved in a tag of the data type BYTE, WORD, or DWORD.

Note

Multi-dimensional ARRAY of BOOL

With the "Merge individual bits into a bit sequence" instruction, the use of a multidimensional ARRAY of BOOL is not permitted.

Note

Length of the ARRAY, STRUCT or PLC data type

The ARRAY, STRUCT or the PLC data type must have exactly the number of elements that is specified by the bit sequence.

This means for the data type BYTE, for example, the ARRAY, anonymous STRUCT or the PLC data type must have exactly 8 elements (WORD = 16, and DWORD = 32).

Note

Availability of the instruction

The instruction can be used with a CPU of the S7-1200 series as of firmware version >4.2, and for a CPU of the S7-1500 series as of firmware version 2.1.

The enable output ENO returns signal state "0" if one of the following conditions applies:

- Enable input EN has signal state "0".
- The ARRAY, anonymous STRUCT or the PLC data type (UDT) has fewer or more BOOL elements than specified by the bit sequence. In this case the BOOL elements are not transferred.
- Fewer than the necessary number of bits are available.

Data types for the GATHER instruction

The following table shows the parameters of the instruction:

Parameter	Declaration	Data type		Memory area	Description
		S7-1200	S7-1500		
EN	Input	BOOL	BOOL	I, Q, M, D, L or constant	Enable input
ENO	Output	BOOL	BOOL	I, Q, M, D, L	Enable output
IN	Input	ARRAY[*] of BOOL, STRUCT or PLC data type *: 8, 16, 32 or 64 elements	ARRAY[*] of BOOL, STRUCT or PLC data type *: 8, 16, 32 or 64 elements	I, Q, M, D, L	ARRAY, STRUCT or PLC data type, the bits of which are merged into a bit sequence. The values must not be located in the I/O area or in the DB of a technology object.
OUT	Output	BYTE, WORD, DWORD	BYTE, WORD, DWORD	I, Q, M, D, L	Merged bit sequence, saved in a tag

You can select the required bit sequence from the "???" drop-down list of the instruction box.

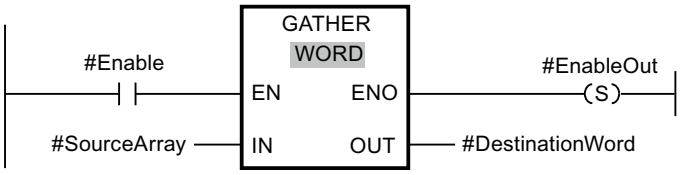
You can find additional information on valid data types under "See also".

Example with an ARRAY

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceArray		ARRAY[0..15] of BOOL
EnableOut	Output	BOOL
DestinationWord		WORD

The following example shows how the instruction works:



The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceArray	The operand "SourceArray" is of the data type ARRAY[0..15] of BOOL. It consists of 16 elements and is therefore just as large as the WORD in which the bits are to be merged.
OUT	DestinationWord	WORD (16 bits)

If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. The bits of the #SourceArray operand are merged into a WORD. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

You can find additional information and the program code for the above-named example here: Sample Library for Instructions.

Example with a PLC data type (UDT)

Create the following PLC data type "myBits":

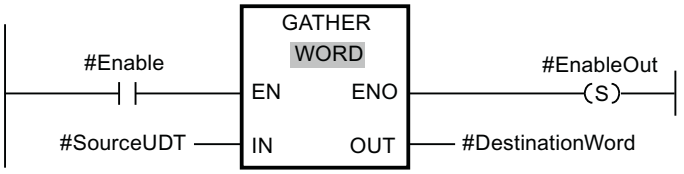
myBits

	Name	Data type
1	GeneralError	Bool
2	DeviceError	Bool
3	CommError	Bool
4	myError1	Bool
5	myError2	Bool
6	myError3	Bool
7	myError4	Bool
8	myError5	Bool
9	myError6	Bool
10	myError7	Bool
11	myError8	Bool
12	myError9	Bool
13	myError10	Bool
14	myError11	Bool
15	myError12	Bool
16	myError13	Bool

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceUDT		"myBits"
EnableOut	Output	BOOL
DestinationWord		WORD

The following example shows how the instruction works:



The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceUDT	The "SourceUDT" operand has the PLC data type (UDT). It consists of 16 elements and is therefore just as large as the WORD in which the bits are to be merged.
OUT	DestinationWord	WORD (16 bits)

If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. The bits of the #SourceUDT operand are merged into a WORD. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

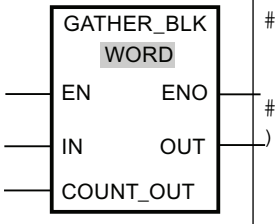
8.6.14 GATHER_BLK

Description

The "Merge individual bits into multiple elements of an ARRAY of bit sequence" instruction merges the bits from an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements into one or multiple elements of an ARRAY of <bit sequence>. At the COUNT_OUT parameter you specify how many elements of the destination ARRAY are going to be written. With this step you also implicitly specify how many elements of the ARRAY of BOOL, the anonymous STRUCT or the PLC data type are required. The destination ARRAY at the OUT parameter may have more elements than specified at the COUNT_OUT parameter. The ARRAY of <bit sequence> must have sufficient elements to save the bits that are going to be merged. However, the destination ARRAY may also be larger.

Table 8-105 GATHER

LAD/FBD	SCL	Description

	`GATHER_BLK(IN := #SourceArrayBool[0],`
	`COUNT_OUT := #CounterOutput,`
	`OUT => #DestinationArrayWord[2]);`
		The "Merge individual bits into multiple elements of an ARRAY of bit sequence" instruction merges the bits from an ARRAY of BOOL, an anonymous STRUCT or a PLC data type exclusively with Boolean elements into one or multiple elements of an ARRAY of <bit sequence>. At the COUNT_OUT parameter you specify how many elements of the destination ARRAY are going to be written. With this step you also implicitly specify how many elements of the ARRAY of BOOL, the anonymous STRUCT or the PLC data type are required. The destination ARRAY at the OUT parameter may have more elements than specified at the COUNT_OUT parameter. The ARRAY of <bit sequence> must have sufficient elements to save the bits that are going to be merged. However, the destination ARRAY may also be larger.

Note

NO data is written when ENO is false

In the S7-1200 CPU only for the GATHER_BLK instruction, If ENO is FALSE, no data is written to the output.

Note

Multi-dimensional ARRAY of BOOL

If the ARRAY is a multi-dimensional ARRAY of BOOL, the filling bits of the dimensions contained are counted as well even if they were not explicitly declared.

Example 1: An ARRAY[1..10,0..4,1..2] of BOOL is handled like an ARRAY[1..10,0..4,1..8] of BOOL or like an ARRAY[0..399] of BOOL.

Example 2: At the OUT parameter, an ARRAY[0..5] of WORD (sourceArrayWord[2]) is interconnected. The COUNT_IN parameter has the value "3". At the IN parameter, an ARRAY[0..1,0..5,0..7] of BOOL (destinationArrayBool[0,0,0]) is interconnected. Both the array at the IN parameter and at the OUT parameter has a size of 96 bits. 48 individual bits are merged from the ARRAY of BOOL.

Note

If the ARRAY low limit of the source ARRAY is not "0", note the following:

For performance reasons the index must always start at a BYTE, WORD, or DWORD limit. This means the index must be calculated starting at the low limit of the ARRAY. The following formula is used as basis for this calculation:

Valid indices = ARRAY low limit + n(number of bit sequences) × number of bits of the desired bit sequence

For an ARRAY[-2..45] of BOOL and the bit sequence WORD the calculation looks as follows:

- Valid index (-2) = -2 + 0 × 16
- Valid index (14) = -2 + 1 × 16
- Valid index (30) = -2 + 2 × 16

You can find an example described below.

Note

Availability of the instruction

The instruction can be used with a CPU of the S7-1200 series as of firmware version >4.2, and for a CPU of the S7-1500 series as of firmware version 2.1.

The enable output ENO returns signal state "0" if one of the following conditions applies:

- Enable input EN has signal state "0".
- The index of the source ARRAY does not start at a BYTE, WORD, or DWORD limit. In this case, no result is written to the ARRAY of <bit sequence>.
- The ARRAY[*] of <bit sequence> does not provide the required number of elements.
  - S7-1500-CPU: In this case as many bit sequences as possible are merged and written to the ARRAY of <bit sequence>. The remaining bits are no longer taken into account.
  - S7-1200-CPU: There is no copying procedure.

Data types for the GATHER_BLK instruction

The following table shows the parameters of the instruction:

Parameter	Declaration	Data type		Memory area	Description
		S7-1200	S7-1500		
EN	Input	BOOL	BOOL	I, Q, M, D, L or constant	Enable input
ENO	Output	BOOL	BOOL	I, Q, M, D, L	Enable output
IN	Input	Element of an ARRAY[*] of BOOL, STRUCT or PLC data type	Element of an ARRAY[*] of BOOL, STRUCT or PLC data type	I, Q, M, D, L	ARRAY of BOOL, STRUCT or PLC data type whose bits are merged (source ARRAY) The values must not be located in the I/O area or in the DB of a technology object.
COUNT_OUT	Input	USINT, UINT, UDINT	USINT, UINT, UDINT	I, Q, M, D, L	Counter how many elements of the target ARRAY are to be described. The value must not be in the I/O area or in the database of a technology object.
OUT	Output	Element of an ARRAY[*] of <bit sequence>	Element of an ARRAY[*] of <bit sequence>	I, Q, M, D, L	ARRAY of <bit sequence> to which the bits are saved (destination ARRAY)

You can select the required bit sequence from the "???" drop-down list of the instruction box.

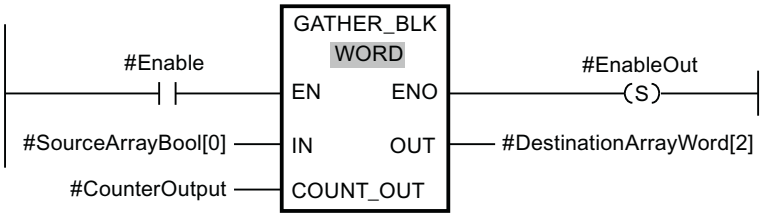
You can find additional information on valid data types under "See also".

Example of a source ARRAY with the low limit "0"

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceArrayBool		ARRAY[0..95] of BOOL
CounterOutput		UDINT
EnableOut	Output	BOOL
DestinationArrayWord		ARRAY[0..5] of WORD

The following example shows how the instruction works:



The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceArrayBool[0]	The operand "SourceArrayBool" is of the data type ARRAY[0..95] of BOOL. This means it provides 96 BOOL elements that can merged into words once again.
COUNT_OUT	CounterOutput = 3	UDINT3 (3 words are to be written. This means 48 bits must be available in the source ARRAY.)
OUT	DestinationArrayWord[2]	The operand "DestinationArrayWord" is of the data type ARRAY[0..5] of WORD. This means 6 WORD elements are available.

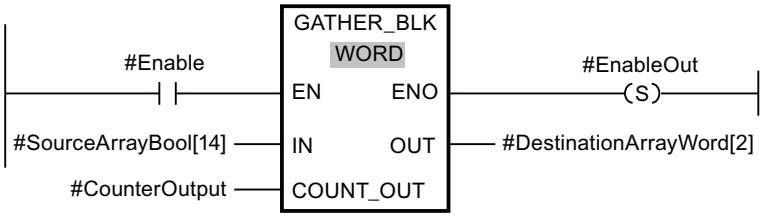
If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. As of the 1st element of the #SourceArrayBool operand, 48 bits are merged in the #DestinationArrayWord operand. The starting point in the destination ARRAY is the 3rd element. This means that the first 16 bits are written into the 3rd word, the second 16 bits into the 4th word and the third 16 bits into the 5th word of the destination ARRAY. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

Example of a source ARRAY with the low limit "-2"

Create the following tags in the block interface:

Tag	Section	Data type
Enable	Input	BOOL
SourceArrayBool		ARRAY[-2..93] of BOOL
CounterOutput		UDINT
EnableOut	Output	BOOL
DestinationArrayWord		ARRAY[0..5] of WORD

The following example shows how the instruction works:



The following table shows how the instruction works using specific operand values:

Parameter	Operand	Data type
IN	SourceArrayBool[14]	The operand "SourceArrayBool" is of the data type ARRAY[-2..93] of BOOL. Because the starting point is the 16th element, only 80 BOOL elements that can be merged into words again are available.
COUNT_OUT	CounterOutput = 3	UDINT3 (3 words are to be written. This means 48 bits must be available in the source ARRAY.)
OUT	DestinationArrayWord[2]	The operand "DestinationArrayWord" is of the data type ARRAY[0..5] of WORD. This means 6 WORD elements are available.

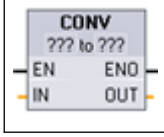
If the operand #Enable returns the signal state "1" at the enable input EN, the instruction is executed. Starting with the 16th element of the #SourceArrayBool operand, 48 bits are merged into the #DestinationArrayWord operand. The starting point in the destination ARRAY is the 3rd element. This means the first 16 bits of the source ARRAY are ignored. The second 16 bits are written into the 3rd word, the third 16 bits into the 4th word and the fourth 16 bits into the 5th word of the destination ARRAY. The remaining 64 bits of the source ARRAY are not taken into account either. If an error occurs during the execution of the instruction, the operand #EnableOut returns the signal state "0" at the enable output ENO.

You can find additional information and the program code for the above-named example here: Sample Library for Instructions.

8.7 Conversion operations

8.7.1 CONV (Convert value)

Table 8-106 Convert (CONV) instruction

LAD / FBD	SCL	Description
	`out := <data type in>_TO_<data type out>(in);`	Converts a data element from one data type to another data type.

[1] For LAD and FBD: Click the "???" and select the data types from the drop-down menu.

[2] For SCL: Construct the conversion instruction by identifying the data type for the input parameter (in) and output parameter (out). For example, DWORD_TO_REAL converts a DWord value to a Real value.

Table 8-107 Data types for the parameters

Parameter	Data type	Description
IN	Bit string[1], SInt, USInt, Int, UInt, DInt, UDInt, Real, LReal, BCD16, BCD32, Char, WChar	Input value
OUT	Bit string[1], SInt, USInt, Int, UInt, DInt, UDInt, Real, LReal, BCD16, BCD32, Char, WChar	Input value converted to a new data type

[1] The instruction does not allow you to select Bit strings (Byte, Word, DWord). To enter an operand of data type Byte, Word, or DWord for a parameter of the instruction, select an unsigned integer with the same bit length. For example, select USInt for a Byte, UInt for a Word, or UDInt for a DWord.

After you select the (convert from) data type, a list of possible conversions is shown in the (convert to) dropdown list. Conversions from and to BCD16 are restricted to the Int data type. Conversions from and to BCD32 are restricted to the DInt data type.

Table 8-108 ENO status

ENO	Description	Result OUT
1	No error	Valid result
0	IN is +/- INF or +/- NaN	+/- INF or +/- NaN
0	Result exceeds valid range for OUT data type	OUT is set to the IN value

8.7.2 Conversion instructions for SCL

Conversion instructions for SCL

Table 8-109 Conversion from a Bool, Byte, Word, or DWord

Data type	Instruction	Result
Bool	**BOOL_TO_BYTE, BOOL_TO_WORD, BOOL_TO_DWORD, BOOL_TO_INT, BOOL_TO_DINT**	The value is transferred to the least significant bit of the target data type.
Byte	**BYTE_TO_BOOL**	The least significant bit is transferred into the destination data type.
	BYTE_TO_WORD, BYTE_TO_DWORD	The value is transferred to the least significant byte of the target data type.
	BYTE_TO_SINT, BYTE_TO_USINT	The value is transferred to the target data type.
	BYTE_TO_INT, BYTE_TO_UINT, BYTE_TO_DINT, BYTE_TO_UDINT	The value is transferred to the least significant byte of the target data type.
Word	**WORD_TO_BOOL**	The least significant bit is transferred into the destination data type.
	WORD_TO_BYTE	The least significant byte of the source value is transferred to the target data type
	WORD_TO_DWORD	The value is transferred to the least significant word of the target data type.
	WORD_TO_SINT, WORD_TO_USINT	The least significant byte of the source value is transferred to the target data type.
	WORD_TO_INT, WORD_TO_UINT	The value is transferred to the target data type.
	WORD_TO_DINT, WORD_TO_UDINT	The value is transferred to the least significant word of the target data type.
DWord	**DWORD_TO_BOOL**	The least significant bit is transferred into the destination data type.
	DWORD_TO_BYTE, DWORD_TO_WORD, DWORD_TO_SINT	The least significant byte of the source value is transferred to the target data type.
	DWORD_TO_USINT, DWORD_TO_INT, DWORD_TO_UINT	The least significant word of the source value is transferred to the target data type.
	DWORD_TO_DINT, DWORD_TO_UDINT, DWORD_TO_REAL	The value is transferred to the target data type.

Table 8-110 Conversion from a short integer (SInt or USInt)

Data type	Instruction	Result
SInt	**SINT_TO_BOOL**	The least significant bit is transferred into the destination data type.
	SINT_TO_BYTE	The value is transferred to the target data type
	SINT_TO_WORD, SINT_TO_DWORD	The value is transferred to the least significant byte of the target data type.
	SINT_TO_INT, SINT_TO_DINT, SINT_TO_USINT, SINT_TO_UINT, SINT_TO_UDINT, SINT_TO_REAL, SINT_TO_LREAL, SINT_TO_CHAR, SINT_TO_STRING	The value is converted.
USInt	**USINT_TO_BOOL**	The least significant bit is transferred into the destination data type.
	USINT_TO_BYTE	The value is transferred to the target data type
	USINT_TO_WORD, USINT_TO_DWORD, USINT_TO_INT, USINT_TO_UINT, USINT_TO_DINT, USINT_TO_UDINT	The value is transferred to the least significant byte of the target data type.
	USINT_TO_SINT, USINT_TO_REAL, USINT_TO_LREAL, USINT_TO_CHAR, USINT_TO_STRING	The value is converted.

Table 8-111 Conversion from an integer (Int or UInt)

Data type	instruction	Result
Int	**INT_TO_BOOL**	The least significant bit is transferred into the destination data type.
	INT_TO_BYTE, INT_TO_DWORD, INT_TO_SINT, INT_TO_USINT, INT_TO_UINT, INT_TO_UDINT, INT_TO_REAL, INT_TO_LREAL, INT_TO_CHAR, INT_TO_STRING	The value is converted.
	INT_TO_WORD	The value is transferred to the target data type.
	INT_TO_DINT	The value is transferred to the least significant byte of the target data type.
UInt	**UINT_TO_BOOL**	The least significant bit is transferred into the destination data type.
	UINT_TO_BYTE, UINT_TO_SINT, UINT_TO_USINT, UINT_TO_INT, UINT_TO_REAL, UINT_TO_LREAL, UINT_TO_CHAR, UINT_TO_STRING	The value is converted.
	UINT_TO_WORD, UINT_TO_DATE	The value is transferred to the target data type.
	UINT_TO_DWORD, UINT_TO_DINT, UINT_TO_UDINT	The value is transferred to the least significant byte of the target data type.

Table 8-112 Conversion from a double integer (Dint or UDInt)

Data type	Instruction	Result
DInt	**DINT_TO_BOOL**	The least significant bit is transferred into the destination data type.
	DINT_TO_BYTE, DINT_TO_WORD, DINT_TO_SINT, DINT_TO_USINT, DINT_TO_INT, DINT_TO_UINT, DINT_TO_UDINT, DINT_TO_REAL, DINT_TO_LREAL, DINT_TO_CHAR, DINT_TO_STRING	The value is converted.
	DINT_TO_DWORD, DINT_TO_TIME	The value is transferred to the target data type.
UDInt	**UDINT_TO_BOOL**	The least significant bit is transferred into the destination data type.
	UDINT_TO_BYTE, UDINT_TO_WORD, UDINT_TO_SINT, UDINT_TO_USINT, UDINT_TO_INT, UDINT_TO_UINT, UDINT_TO_DINT, UDINT_TO_REAL, UDINT_TO_LREAL, UDINT_TO_CHAR, UDINT_TO_STRING	The value is converted.
	UDINT_TO_DWORD, UDINT_TO_TOD	The value is transferred to the target data type.

Table 8-113 Conversion from a Real number (Real or LReal)

Data type	Instruction	Result
Real	**REAL_TO_DWORD, REAL_TO_LREAL**	The value is transferred to the target data type.
	REAL_TO_SINT, REAL_TO_USINT, REAL_TO_INT, REAL_TO_UINT, REAL_TO_DINT, REAL_TO_UDINT, REAL_TO_STRING	The value is converted.
LReal	**LREAL_TO_SINT, LREAL_TO_USINT, LREAL_TO_INT, LREAL_TO_UINT, LREAL_TO_DINT, LREAL_TO_UDINT, LREAL_TO_REAL, LREAL_TO_STRING**	The value is converted.

Table 8-114 Conversion from Time, DTL, TOD or Date

Data type	Instruction	Result
Time	**TIME_TO_DINT**	The value is transferred to the target data type.
DTL	**DTL_TO_DATE, DTL_TO_TOD**	The value is converted.
TOD	**TOD_TO_UDINT**	The value is converted.
Date	**DATE_TO_UINT**	The value is converted.

Table 8-115 Conversion from a Char or String

Data type	Instruction	Result
Char	**CHAR_TO_SINT, CHAR_TO_USINT, CHAR_TO_INT, CHAR_TO_UINT, CHAR_TO_DINT, CHAR_TO_UDINT**	The value is converted.
	CHAR_TO_STRING	The value is transferred to the first character of the string.

Data type	Instruction	Result
String	`STRING_TO_SINT, STRING_TO_USINT, STRING_TO_INT, STRING_TO_UINT, STRING_TO_DINT, STRING_TO_UDINT, STRING_TO_REAL, STRING_TO_LREAL`	The value is converted.
	`STRING_TO_CHAR`	The first character of the string is copied to the Char.

8.7.3 ROUND (Round numerical value) and TRUNC (Truncate numerical value)

Table 8-116 ROUND and TRUNC instructions

LAD / FBD	SCL	Description
	`out := ROUND (in);`	Converts a real number to an integer. For LAD/FBD, you click the "???" in the instruction box to select the data type for the output, for example "DInt". For SCL, the default data type for the output of the ROUND instruction is DINT. To round to another output data type, enter the instruction name with the explicit name of the data type, for example, ROUND_REAL or ROUND_LREAL. The real number fraction is rounded to the nearest integer value (IEEE - round to nearest). If the number is exactly one-half the span between two integers (for example, 10.5), then the number is rounded to the even integer. For example: • ROUND (10.5) = 10 • ROUND (11.5) = 12
	`out := TRUNC(in);`	TRUNC converts a real number to an integer. The fractional part of the real number is truncated to zero (IEEE - round to zero).

[1] For LAD and FBD: Click the "???" (by the instruction name) and select a data type from the drop-down menu.

Table 8-117 Data types for the parameters

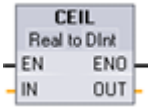
Parameter	Data type	Description
IN	Real, LReal	Floating point input
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal	Rounded or truncated output

Table 8-118 ENO status

ENO	Description	Result OUT
1	No error	Valid result
0	IN is +/- INF or +/- NaN	+/- INF or +/- NaN

8.7.4 CEIL and FLOOR (Generate next higher and lower integer from floating-point number)

Table 8-119 CEIL and FLOOR instructions

LAD / FBD	SCL	Description
	`out := CEIL(in);`	Converts a real number (Real or LReal) to the closest integer greater than or equal to the selected real number (IEEE "round to +infinity").
	`out := FLOOR(in);`	Converts a real number (Real or LReal) to the closest integer smaller than or equal to the selected real number (IEEE "round to -infinity").

[1] For LAD and FBD: Click the "???" (by the instruction name) and select a data type from the drop-down menu.

Table 8-120 Data types for the parameters

Parameter	Data type	Description
IN	Real, LReal	Floating point input
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal	Converted output

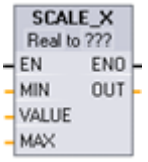
Table 8-121 ENO status

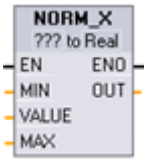
ENO	Description	Result OUT
1	No error	Valid result
0	IN is +/- INF or +/- NaN	+/- INF or +/- NaN

8.7.5 SCALE_X (Scale) and NORM_X (Normalize)

Table 8-122 SCALE_X and NORM_X instructions

LAD / FBD	SCL	Description

	`out :=SCALE_X(min:=_in_,`
	`value:=_in_,`
	`max:=_in_);`
		Scales the normalized real parameter VALUE where (0.0 <= VALUE <= 1.0) in the data type and value range specified by the MIN and MAX parameters:
		OUT = VALUE (MAX - MIN) + MIN

	`out :=NORM_X(min:=_in_,`
	`value:=_in_,`
	`max:=_in_);`
		Normalizes the parameter VALUE inside the value range specified by the MIN and MAX parameters:
		OUT = (VALUE - MIN) / (MAX - MIN),
		where (0.0 <= OUT <= 1.0)

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-123 Data types for the parameters

Parameter	Data type[1]	Description
MIN	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal	Input minimum value for range
VALUE	SCALE_X: Real, LReal NORM_X: SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal	Input value to scale or normalize
MAX	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal	Input maximum value for range
OUT	SCALE_X: SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal NORM_X: Real, LReal	Scaled or normalized output value

[1] For SCALE_X: Parameters MIN, MAX, and OUT must be the same data type.
For NORM_X: Parameters MIN, VALUE, and MAX must be the same data type.

Note

SCALE_X parameter VALUE should be restricted to (0.0 <= VALUE <= 1.0)

If parameter VALUE is less than 0.0 or greater than 1.0:

- The linear scaling operation can produce OUT values that are less than the parameter MIN value or above the parameter MAX value for OUT values that fit within the value range of the OUT data type. SCALE_X execution sets ENO = TRUE for these cases.
- It is possible to generate scaled numbers that are not within the range of the OUT data type. For these cases, the parameter OUT value is set to an intermediate value equal to the least-significant portion of the scaled real number prior to final conversion to the OUT data type. SCALE_X execution sets ENO = FALSE in this case.

NORM_X parameter VALUE should be restricted to (MIN <= VALUE <= MAX)

If parameter VALUE is less than MIN or greater than MAX, the linear scaling operation can produce normalized OUT values that are less than 0.0 or greater than 1.0. NORM_X execution sets ENO = TRUE in this case.

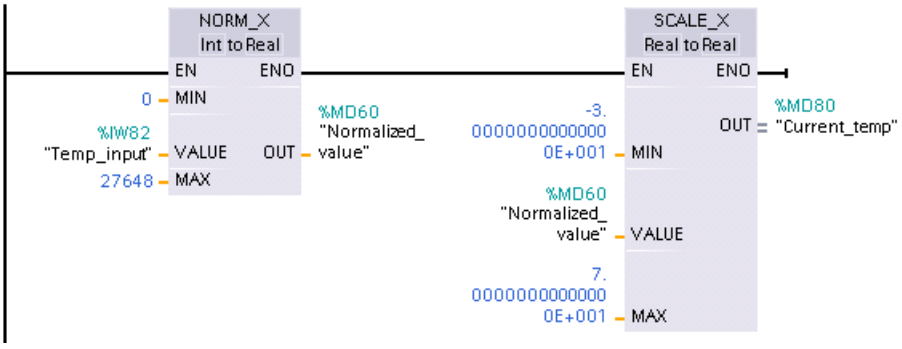
Table 8-124 ENO status

ENO	Condition	Result OUT
1	No error	Valid result
0	Result exceeds valid range for the OUT data type	Intermediate result: The least-significant portion of a real number prior to final conversion to the OUT data type.
0	Parameters MAX <= MIN	SCALE_X: The least-significant portion of the Real number VALUE to fill up the OUT size. NORM_X: VALUE in VALUE data type extended to fill a double word size.
0	Parameter VALUE = +/- INF or +/- NaN	VALUE is written to OUT

Example (LAD): normalizing and scaling an analog input value

An analog input from an analog signal module or signal board using input in current is in the range 0 to 27648 for valid values. Suppose an analog input represents a temperature where the 0 value of the analog input represents -30.0 degrees C and 27648 represents 70.0 degrees C.

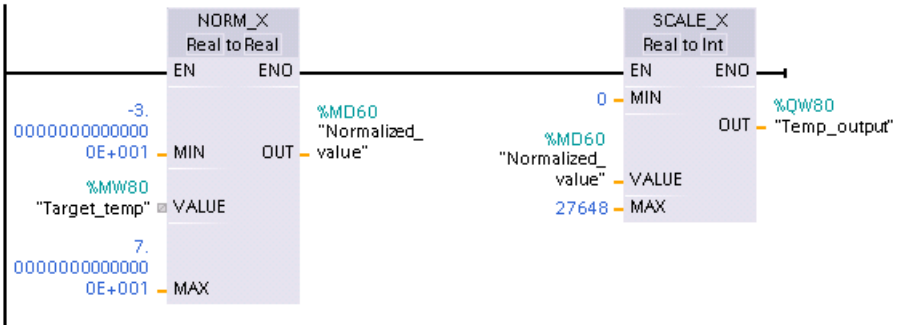
To transform the analog value to the corresponding engineering units, normalize the input to a value between 0.0 and 1.0, and then scale it between -30.0 and 70.0. The resulting value is the temperature represented by the analog input in degrees C:



Note that if the analog input was from an analog signal module or signal board using voltage, the MIN value for the NORM_X instruction would be -27648 instead of 0.

Example (LAD): normalizing and scaling an analog output value

An analog output to be set in an analog signal module or signal board using output in current must be in the range 0 to 27648 for valid values. Suppose an analog output represents a temperature setting where the 0 value of the analog input represents -30.0 degrees C and 27648 represents 70.0 degrees C. To convert a temperature value in memory that is between -30.0 and 70.0 to a value for the analog output in the range 0 to 27648, you must normalize the value in engineering units to a value between 0.0 and 1.0, and then scale it to the range of the analog output, 0 to 27648:



Note that if the analog output was for an analog signal module or signal board using voltage, the MIN value for the SCALE_X instruction would be -27648 instead of 0.

Additional information on analog input representations (Page 1285) and analog output representations (Page 1286) in both voltage and current can be found in the Technical Specifications.

8.7.6 Variant conversion instructions

8.7.6.1 VARIANT_TO_DB_ANY (Convert VARIANT to DB_ANY)

You use the "VARIANT to DB_ANY" instruction to read the operand at the IN parameter and convert it to the data type DB_ANY. The IN parameter is of the Variant data type and represents either an instance data block or an ARRAY data block. When you create the program, you do not need to know which data block corresponds to the IN parameter. The instruction reads the data block number during runtime and writes it to the operand at the RET_VAL parameter.

Table 8-125 VARIANT_TO_DB_ANY instruction

LAD / FBD	SCL	Description
Not available	`RET_VAL := VARIANT_TO_DB_ANY( in := _variant_in_, err => _int_out_);`	Reads the operand from the Variant IN parameter and stores it in the function result, which is of the type DB_ANY

Table 8-126 Parameters for the VARIANT_TO_DB_ANY instruction

Parameter	Data type	Description
IN	Variant	Variant that represents and instance data block or an array data block
RET_VAL	DB_ANY	Output DB_ANY data type that contains the converted data block number
ERR	Int	Error information

Table 8-127 ENO status

ENO	Condition	Result
1	No error	Instruction converts the input Variant and stores it in the DB_ANY function output
0	Enable input EN has the signal state "0" or the IN parameter is invalid.	Instruction does nothing.

Table 8-128 Error output codes for the VARIANT_TO_DB_ANY instruction

Err (W#16#...)	Description
0000	No error
252C	The Variant data type at IN parameter has the value 0. The CPU changes to STOP mode.
8131	The data block does not exist or is too short (first access).
8132	The data block is too short and not an Array data block (second access).
8134	The data block is write-protected
8150	The data type Variant at parameter IN provides the value "0". To receive this error message, the "Handle errors within block" block property must be activated. Otherwise the CPU changes to STOP mode and sends the error code 16#252C
8154	The data block has the incorrect data type.
*You can display error codes in the program editor as integer or hexadecimal values.	

8.7.6.2 DB_ANY_TO_VARIANT (Convert DB_ANY to VARIANT)

You use the "DB_ANY to VARIANT" instruction to read the number of a data block that meets the requirements listed below. The operand at the IN parameter has the data type DB_ANY, which means you do not need to know during program creation which data block is to be read. The instruction reads the data block number during runtime and writes it to the function result RET_VAL by means of a VARIANT pointer.

Table 8-129 DB_ANY_TO_VARIANT instruction

LAD / FBD	SCL	Description

Not available	`RET_VAL := DB_ANY_TO_VARIANT(`
	`  in := _db_any_in_,`
	`  err => _int_out_);`
		Reads the data block number from the Variant IN parameter and stores it in the function result, which is of the type Variant

Table 8-130 Parameters for the DB_ANY_TO_VARIANT instruction

Parameter	Data type	Description
IN	DB_ANY	Variant that contains the data block number
RET_VAL	Variant	Output DB_ANY data type that contains the converted data block number
ERR	Int	Error information

Table 8-131 ENO status

ENO	Condition	Result
1	No error	Instruction converts the data block number in the variant and stores it in the function DB_ANY output
0	Enable input EN has the signal state "0" or the IN parameter is invalid.	Instruction does nothing.

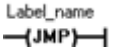
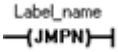
Table 8-132 Error output codes for the DB_ANY_TO_VARIANT instruction

Err (W#16#...)	Description
0000	No error
8130	The number of the data block is 0.
8131	The data block does not exist or is too short.
8132	The data block is too short and not an Array data block.
8134	The data block is write-protected.
8154	The data block has the incorrect data type.
8155	Unknown type code
*You can display error codes in the program editor as integer or hexadecimal values.	

8.8 Program control operations

8.8.1 JMP (Jump if RLO = 1), JMPN (Jump if RLO = 0), and Label (Jump label) instructions

Table 8-133 JMP, JMPN, and LABEL instruction

LAD	FBD	SCL	Description
		See the GOTO (Page 301) statement.	Jump if RLO (result of logic operation) = 1: If there is power flow to a JMP coil (LAD), or if the JMP box input is true (FBD), then program execution continues with the first instruction following the specified label.
			Jump if RLO = 0: If there is no power flow to a JMPN coil (LAD), or if the JMPN box input is false (FBD), then program execution continues with the first instruction following the specified label.
			Destination label for a JMP or JMPN jump instruction.

[1] You create your label names by typing in the LABEL instruction directly. Use the parameter helper icon to select the available label names for the JMP and JMPN label name field. You can also type a label name directly into the JMP or JMPN instruction.

Table 8-134 Data types for the parameters

Parameter	Data type	Description
Label_name	Label identifier	Identifier for Jump instructions and the corresponding jump destination program label

- Each label must be unique within a code block.
- You can jump within a code block, but you cannot jump from one code block to another code block.
- You can jump forward or backward.
- You can jump to the same label from more than one place in the same code block.

8.8.2 JMP_LIST (Define jump list)

Table 8-135 JMP_LIST instruction

LAD / FBD	SCL	Description

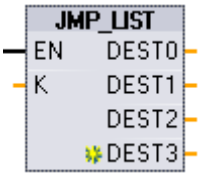
	`CASE k OF`
	`0: GOTO dest0;`
	`1: GOTO dest1;`
	`2: GOTO dest2;`
	`[n: GOTO destn;]`
	`END_CASE;`
		The JMP_LIST instruction acts as a program jump distributor to control the execution of program sections. Depending on the value of the K input, a jump occurs to the corresponding program label. Program execution continues with the program instructions that follow the destination jump label. If the value of the K input exceeds the number of labels - 1, then no jump occurs and processing continues with the next program network.

Table 8-136 Data types for parameters

Parameter	Data type	Description
K	UInt	Jump distributor control value
DEST0, DEST1, .., DESTn.	Program Labels	Jump destination labels corresponding to specific K parameter values: If the value of K equals 0, then a jump occurs to the program label assigned to the DEST0 output. If the value of K equals 1, then a jump occurs to the program label assigned to the DEST1 output, and so on. If the value of the K input exceeds the (number of labels - 1), then no jump occurs and processing continues with the next program network.

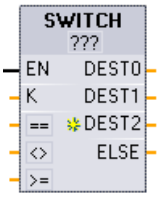
For LAD and FBD: When the JMP_LIST box is first placed in your program, there are two jump label outputs. You can add or delete jump destinations.

Click the create icon inside the box (on the left of the last DEST parameter) to add new outputs for jump labels.

- Right-click on an output stub and select the "Insert output" command.
- Right-click on an output stub and select the "Delete" command.

8.8.3 SWITCH (Jump distributor)

Table 8-137 SWITCH instruction

LAD / FBD	SCL	Description
	`Not available`	The SWITCH instruction acts as a program jump distributor to control the execution of program sections. Depending on the result of comparisons between the value of the K input and the values assigned to the specified comparison inputs, a jump occurs to the program label that corresponds to the first comparison test that is true. If none of the comparisons is true, then a jump to the label assigned to ELSE occurs. Program execution continues with the program instructions that follow the destination jump label.

[1] For LAD and FBD: Click below the box name and select a data type from the drop-down menu.

[2] For SCL: Use an IF-THEN set of comparisons.

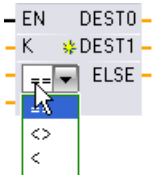
Table 8-138 Data types for parameters

Parameter	Data type[1]	Description
K	UInt	Common comparison value input
==, <>, <, <=, >. >=	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, TOD, Date	Separate comparison value inputs for specific comparison types
DEST0, DEST1, .., DESTn, ELSE	Program Labels	Jump destination labels corresponding to specific comparisons: The comparison input below and next to the K input is processed first and causes a jump to the label assigned to DEST0, if the comparison between the K value and this input is true. The next comparison test uses the next input below and causes a jump to the label assigned to DEST1, if the comparison is true, The remaining comparisons are processed similarly and if none of the comparisons are true, then a jump to the label assigned to the ELSE output occurs.

[1] The K input and comparison inputs (==, <>, <, <=, >, >=) must be the same data type.

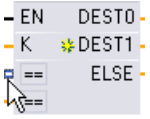
Adding inputs, deleting inputs, and specifying comparison types

When the LAD or FBD SWITCH box is first placed in your program there are two comparison inputs. You can assign comparison types and add inputs/jump destinations, as shown below.



Click a comparison operator inside the box and select a new operator from the drop-down list.

Click the create icon inside the box (to the left of the last DEST parameter) to add new comparison-destination parameters.



- Right-click on an input stub and select the "Insert input" command.
- Right-click on an input stub and select the "Delete" command.

Table 8-139 SWITCH box data type selection and allowed comparison operations

Data type	Comparison	Operator syntax
Byte, Word, DWord	Equal	==
	Not equal	<>
SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Time, TOD, Date	Equal	==
	Not equal	<>
	Greater than or equal	>=
	Less than or equal	<=
	Greater than	>
	Less than	<

SWITCH box placement rules

- No LAD/FBD instruction connection in front of the compare input is allowed.
- There is no ENO output, so only one SWITCH instruction is allowed in a network and the SWITCH instruction must be the last operation in a network.

8.8.4 RET (Return)

The optional RET instruction is used to terminate the execution of the current block. If and only if there is power flow to the RET coil (LAD) or if the RET box input is true (FBD), then program execution of the current block will end at that point and instructions beyond the RET instruction will not be executed. If the current block is an OB, the "Return_Value" parameter is ignored. If the current block is a FC or FB, the value of the "Return_Value" parameter is passed back to the calling routine as the ENO value of the called box.

You are not required to use a RET instruction as the last instruction in a block; this is done automatically for you. You can have multiple RET instructions within a single block.

For SCL, see the RETURN (Page 301) statement.

Table 8-140 Return_Value (RET) execution control instruction

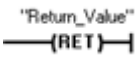
LAD	FBD	SCL	Description
		`RETURN;`	Terminates the execution of the current block

Table 8-141 Data types for the parameters

Parameter	Data type	Description
Return_Value	Bool	The "Return_value" parameter of the RET instruction is assigned to the ENO output of the block call box in the calling block.

Sample steps for using the RET instruction inside an FC code block:

1. Create a new project and add an FC:
2. Edit the FC:
   - Add instructions from the instruction tree.
   - Add a RET instruction, including one of the following for the "Return_Value" parameter: TRUE, FALSE, or a memory location that specifies the required return value.
   - Add more instructions.
3. Call the FC from MAIN [OB1].

The EN input on the FC box in the MAIN code block must be true to begin execution of the FC.

The value specified by the RET instruction in the FC will be present on the ENO output of the FC box in the MAIN code block following execution of the FC for which power flow to the RET instruction is true.

8.8.5 ENDIS_PW (Enable/disable CPU passwords)

Table 8-142 ENDIS_PW instruction

LAD / FBD	SCL	Description

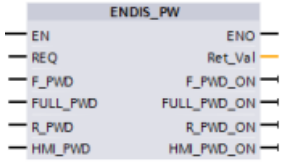
	`ENDIS_PW(`
	`req:=_bool_in_,`
	`f_pwd:=_bool_in_,`
	`full_pwd:=_bool_in_,`
	`r_pwd:=_bool_in_,`
	`hmi_pwd:=_bool_in_,`
	`f_pwd_on=>_bool_out_,`
	`full_pwd_on=>_bool_out_,`
	`r_pwd_on=>_bool_out_,`
	`hmi_pwd_on=>_bool_out_);`
		The ENDIS_PW instruction can allow and disallow client connections to a S7-1200 CPU, even when the client can provide the correct password.
		This instruction does not disallow Web server passwords.

Table 8-143 Data types for the parameters

Parameter and type		Data type	Description
REQ	IN	Bool	Perform function if REQ=1
F_PWD	IN	Bool	Fail-safe password: Allow (=1) or disallow (=0)
FULL_PWD	IN	Bool	Full access password: Allow (=1) or disallow (=0) full access password
R_PWD	IN	Bool	Read access password: Allow (=1) or disallow (=0)
HMI_PWD	IN	Bool	HMI password: Allow (=1) or disallow (=0)
F_PWD_ON	OUT	Bool	Fail-safe password status: Allowed (=1) or disallowed (=0)
FULL_PWD_ON	OUT	Bool	Full access password status: Allowed (=1) or disallowed (=0)
R_PWD_ON	OUT	Bool	Read only password status: Allowed (=1) or disallowed (=0)
HMI_PWD_ON	OUT	Bool	HMI password status: Allowed (=1) or disallowed (=0)
Ret_Val	OUT	Word	Function result

Calling ENDIS_PW with REQ=1 disallows password types where the corresponding password input parameter is FALSE. Each password type can be allowed or disallowed independently. For example, if the fail-safe password is allowed and all other passwords disallowed, then you can restrict CPU access to a small group of employees.

ENDIS_PW is executed synchronously in a program scan and the password output parameters always show the current state of password allowance independent of the input parameter REQ. All passwords that you set to allow must be changeable to disallowed/allowed. Otherwise, an error is returned and all passwords are allowed that were allowed before ENDIS_PW execution.

This means that in a standard CPU (where the fail-safe password is not configured) F_PWD must always be set to 1, to result in a return value of 0. In this case, F_PWD_ON is always 1.

Note

- ENDIS_PW execution can block the access of HMI devices, if the HMI password is disallowed.
- Client sessions that were authorized prior to ENDIS_PW execution may be terminated by ENDIS_PW execution dependent on the existing legitimization level. For example, a connection legitimized with READ password protection will be aborted by ENDIS_PW (REQ=1, R_PWD=0). Other lower protection level connections are also aborted under this scenario. Accordingly, connections legitimized with FULL access are retained.

After a power-up, CPU access is restricted by passwords previously defined in the regular CPU protection configuration. The ability to disallow a valid password must be re-established with a new ENDIS_PW execution. However, if ENDIS_PW is immediately executed and necessary passwords are disallowed, then TIA portal access can be locked out. You can use a timer instruction to delay ENDIS_PW execution and allow time to enter passwords, before the passwords become disallowed.

Note

Restoring a CPU that locks out TIA portal communication

Refer to the "Recovery from a lost password (Page 126)" topic for details about how to erase the internal load memory of a PLC using a memory card.

An operating mode change to STOP caused by errors, STP execution or STEP 7 does not abolish the protection. The protection is valid until the CPU is power cycled. See the following table for details.

Action	Operating mode	ENDIS_PW password control
After memory reset from STEP 7	STOP	Active: Disallowed passwords remain disallowed.
After powering on, or changing a memory card	STOP	Off: No passwords are disallowed.
After ENDIS_PW execution in a program cycle or startup OB	STARTUP, RUN	Active: Passwords are disallowed according to ENDIS_PW parameters
After change of the operating mode from RUN or STARTUP to STOP through STP instruction, error, or STEP 7	STOP	Active: Disallowed passwords remain disallowed

⚠ WARNING

Unauthorized access to a protected CPU

Users with CPU full access or full access (incl. fail-safe) privileges have privileges to read and write PLC variables. Regardless of the access level for the CPU, Web server users can have privileges to read and write PLC variables. Unauthorized access to the CPU or changing PLC variables to invalid values could disrupt process operation and could result in death, severe personal injury and/or property damage.

Authorized users can perform operating mode changes, writes to PLC data, and firmware updates. Siemens recommends that you observe the following security practices:

- Password protect CPU access levels (Page 152) and Web server user IDs (Page 808) with strong passwords.
- Strong passwords are at least twelve characters, are not trivial or easy to guess, and include at least three of the following:
  - Uppercase letters
  - Lowercase letters
  - Digits
  - Special characters
- A trivial password is one that is easy to guess. It is usually based on a characteristic of the user, such as a pet's name, family name, or the company where the user works. For example: Siemens1$, June2015, or Qwerty1234.
- Best practices for generating strong but easy-to-remember passwords include the use of meaningless short sentences and mixing several random words. For example: PC;House#R3d
- Enable access to the Web server only with the HTTPS protocol.
- Do not extend the default minimum privileges of the Web server "Everybody" user.
- Perform error-checking and range-checking on your variables in your program logic because Web page users can change PLC variables to invalid values.
- Use a secure Virtual Private Network (VPN) to connect to the S7-1200 PLC Web server from a location outside your protected network.

Table 8-144 Condition codes

RET_VAL (W#16#...)	Description
0000	No error
8090	The instruction is not supported.
80D0	The password for fail-safe is not configured.
80D1	The password for read/write access is not configured.
80D2	The password for read access is not configured.
80D3	The password for HMI access is not configured.

8.8.6 RE_TRIGR (Restart cycle monitoring time)

Table 8-145 RE_TRIGR instruction

LAD / FBD	SCL	Description
	`RE_TRIGR();`	RE_TRIGR (Re-trigger scan time watchdog) is used to extend the maximum time allowed before the scan cycle watchdog timer generates an error.

Use the RE_TRIGR instruction to restart the scan cycle monitoring timer during a single scan cycle. This has the effect of extending the allowed maximum scan cycle time by one maximum cycle time period, from the last execution of the RE_TRIGR function.

Note

Prior to S7-1200 CPU firmware version 2.2, RE_TRIGR was restricted to execution from a program cycle OB and could be used to extend the PLC scan time indefinitely. ENO = FALSE and the watchdog timer is not reset when RE_TRIGR was executed from a start up OB, an interrupt OB, or an error OB.

For firmware version 2.2 and later, RE_TRIGR can be executed from any OB (including start up, interrupt, and error OBs). However, the PLC scan can only be extended by a maximum of 10x the configured maximum cycle time.

Setting the PLC maximum cycle time

Configure the value for maximum scan cycle time in the Device configuration for "Cycle time".

Table 8-146 Cycle time values

Cycle time monitor	Minimum value	Maximum value	Default value
Maximum cycle time	1 ms	6000 ms	150 ms

Watchdog timeout

If the maximum scan cycle timer expires before the scan cycle has been completed, an error is generated. If the user program includes a time error interrupt OB (OB 80), the CPU executes the time error interrupt OB, which can include program logic to create a special reaction.

If the user program does not include a time error interrupt OB, the first timeout condition is ignored and the CPU remains in RUN mode. If a second maximum scan time timeout occurs in the same program scan (2 times the maximum cycle time value), then an error is triggered that causes a transition to STOP mode.

In STOP mode, your program execution stops while CPU system communications and system diagnostics continue.

8.8.7 STP (Exit program)

Table 8-147 STP instruction

LAD / FBD	SCL	Description
	`STP();`	STP puts the CPU in STOP mode. When the CPU is in STOP mode, the execution of your program and physical updates from the process image are stopped.

For more information see: Configuring the outputs on a RUN-to-STOP transition (Page 93).

If EN = TRUE, then the CPU goes to STOP mode, the program execution stops, and the ENO state is meaningless. Otherwise, EN = ENO = 0.

8.8.8 GET_ERROR and GET_ERROR_ID (Get error and error ID locally) instructions

The get error instructions provide information about program block execution errors. If you add a GET_ERROR or GET_ERROR_ID instruction to your code block, you can handle program errors within your program block.

GET_ERROR

Table 8-148 GET_ERROR instruction

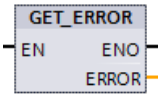
LAD / FBD	SCL	Description
	`GET_ERROR(_out_);`	Indicates that a local program block execution error has occurred and fills a predefined error data structure with detailed error information.

Table 8-149 Data types for the parameters

Parameter	Data type	Description
ERROR	ErrorStruct	Error data structure: You can rename the structure, but not the members within the structure.

Table 8-150 Elements of the ErrorStruct data structure

Structure components	Data type	Description
ERROR_ID	Word	Error ID
FLAGS	Byte	Shows if an error occurred during a block call. • 16#01: Error during a block call. • 16#00: No error during a block call.

<table>
<tr><th colspan="2">Structure components</th><th>Data type</th><th>Description</th></tr>
<tr><td colspan="2">REACTION</td><td>Byte</td><td>Default reaction:
• 0: Ignore (write error),
• 1: Continue with substitute value "0" (read error),
• 2: Skip instruction (system error)</td></tr>
<tr><td colspan="2">CODE_ADDRESS</td><td>CREF</td><td>Information about the address and type of block</td></tr>
<tr><td></td><td>BLOCK_TYPE</td><td>Byte</td><td>Type of block where the error occurred:
• 1: OB
• 2: FC
• 3: FB</td></tr>
<tr><td></td><td>CB_NUMBER</td><td>UInt</td><td>Number of the code block</td></tr>
<tr><td></td><td>OFFSET</td><td>UDInt</td><td>Reference to the internal memory</td></tr>
<tr><td colspan="2">MODE</td><td>Byte</td><td>Access mode: Depending on the type of access, the following information can be output:
<table>
<tr><th>Mode</th><th>(A)</th><th>(B)</th><th>(C)</th><th>(D)</th><th>(E)</th></tr>
<tr><td>0</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>1</td><td></td><td></td><td></td><td></td><td>Offset</td></tr>
<tr><td>2</td><td></td><td></td><td>Area</td><td></td><td></td></tr>
<tr><td>3</td><td>Location</td><td>Scope</td><td></td><td>Number</td><td></td></tr>
<tr><td>4</td><td></td><td></td><td>Area</td><td></td><td>Offset</td></tr>
<tr><td>5</td><td></td><td></td><td>Area</td><td>DB no.</td><td>Offset</td></tr>
<tr><td>6</td><td>PtrNo. /Acc</td><td></td><td>Area</td><td>DB no.</td><td>Offset</td></tr>
<tr><td>7</td><td>PtrNo. / Acc</td><td>Slot No. / Scope</td><td>Area</td><td>DB no.</td><td>Offset</td></tr>
</table></td></tr>
<tr><td colspan="2">OPERAND_NUMBER</td><td>UInt</td><td>Operand number of the machine command</td></tr>
<tr><td colspan="2">POINTER_NUMBER_LOCATION</td><td>UInt</td><td>(A) Internal pointer</td></tr>
<tr><td colspan="2">SLOT_NUMBER_SCOPE</td><td>UInt</td><td>(B) Storage area in internal memory</td></tr>
<tr><td colspan="2">DATA_ADDRESS</td><td>NREF</td><td>Information about the address of an operand</td></tr>
<tr><td></td><td>AREA</td><td>Byte</td><td>(C) Memory area:
• L: 16#40 – 4E, 86, 87, 8E, 8F, C0 – CE
• I: 16#81
• Q: 16#82
• M: 16#83
• DB: 16#84, 85, 8A, 8B</td></tr>
<tr><td></td><td>DB_NUMBER</td><td>UInt</td><td>(D) Number of the data block</td></tr>
<tr><td></td><td>OFFSET</td><td>UDInt</td><td>(E) Relative address of the operand</td></tr>
</table>

GET_ERROR_ID

Table 8-151 GetErrorID instruction

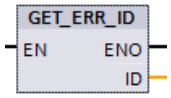
LAD / FBD	SCL	Description
	`GET_ERR_ID();`	Indicates that a program block execution error has occurred and reports the ID (identifier code) of the error.

Table 8-152 Data types for the parameters

Parameter	Data type	Description
ID	Word	Error identifier values for the ErrorStruct ERROR_ID member

Table 8-153 Error_ID values

ERROR_ID hexa-decimal	ERROR_ID decimal	Program block execution error
0	0	No error
2520	9504	Corrupted string
2522	9506	Operand out of range read error
2523	9507	Operand out of range write error
2524	9508	Invalid area read error
2525	9509	Invalid area write error
2528	9512	Data alignment read error (incorrect bit alignment)
2529	9513	Data alignment write error (incorrect bit alignment)
252C	9516	Uninitialized pointer error
2530	9520	DB write protected
2533	9523	Invalid pointer used
2538	9528	Access error: DB does not exist
2539	9529	Access error: Wrong DB used
253A	9530	Global DB does not exist
253C	9532	Wrong version or FC does not exist
253D	9533	Instruction does not exist
253E	9534	Wrong version or FB does not exist
253F	9535	Instruction does not exist
2550	9552	Access error: DB does not exist
2575	9589	Program nesting depth error
2576	9590	Local data allocation error
2942	10562	Physical input point does not exist
2943	10563	Physical output point does not exist

Operation

By default, the CPU responds to a block execution error by logging an error in the diagnostics buffer. However, if you place one or more GET_ERROR or GET_ERROR_ID instructions within a code block, this block is now set to handle errors within the block. In this case, the CPU does not log an error in the diagnostics buffer. Instead, the error information is reported in the output of the GET_ERROR or GET_ERROR_ID instruction. You can read the detailed error information with the GET_ERROR instruction, or read just the error identifier with GET_ERROR_ID instruction. Normally the first error is the most important, with the following errors only consequences of the first error.

The first execution of a GET_ERROR or GET_ERROR_ID instruction within a block returns the first error detected during block execution. This error could have occurred anywhere between the start of the block and the execution of either GET_ERROR or GET_ERROR_ID. Subsequent executions of either GET_ERROR or GET_ERROR_ID return the first error since the previous execution of GET_ERROR or GET_ERROR_ID. The history of errors is not saved, and execution of either instruction will re-arm the PLC system to catch the next error.

The ErrorStruct data type used by the GET_ERROR instruction can be added in the data block editor and block interface editors, so your program logic can access these values. Select ErrorStruct from the data type drop-down list to add this structure. You can create multiple ErrorStruct elements by using unique names. The members of an ErrorStruct cannot be renamed.

Error condition indicated by ENO

If EN = TRUE and GET_ERROR or GET_ERROR_ID executes, then:

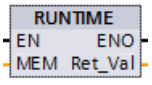
- ENO = TRUE indicates a code block execution error occurred and error data is present
- ENO = FALSE indicates no code block execution error occurred

You can connect error reaction program logic to ENO which activates after an error occurs. If an error exists, then the output parameter stores the error data where your program has access to it.

GET_ERROR and GET_ERROR_ID can be used to send error information from the currently executing block (called block) to a calling block. Place the instruction in the last network of the called block program to report the final execution status of the called block.

8.8.9 RUNTIME (Measure program runtime)

Table 8-154 RUNTIME instruction

LAD / FBD	SCL	Description
	`Ret_Val := RUNTIME( _lread_inout_);`	Measures the runtime of the entire program, individual blocks, or command sequences.

If you want to measure the runtime of your entire program, call the instruction "Measure program runtime" in OB 1. Measurement of the runtime is started with the first call and the output RET_VAL returns the runtime of the program after the second call. The measured runtime includes all CPU processes that can occur during the program execution, for example,

interruptions caused by higher-level events or communication. The instruction "Measure program runtime" reads an internal counter of the CPU and write the value to the IN-OUT parameter MEM. The instruction calculates the current program runtime according to the internal counter frequency and writes it to output RET_VAL.

If you want to measure the runtime of individual blocks or individual command sequences, you need three separate networks. Call the instruction "Measure program runtime" in an individual network within your program. You set the starting point of the runtime measurement with this first call of the instruction. Then you call the required program block or the command sequence in the next network. In another network, call the "Measure program runtime" instruction a second time and assign the same memory to the IN-OUT parameter MEM as you did during the first call of the instruction. The "Measure program runtime" instruction in the third network reads an internal CPU counter and calculates the current runtime of the program block or the command sequence according to the internal counter frequency and writes it to the output RET_VAL.

The Measure program runtime" instruction uses an internal high-frequency counter to calculate the time. If the counter overruns, the instruction returns values <= 0.0. Ignore these runtime values.

Note

The CPU cannot exactly determine the runtime of a command sequence, because the sequence of instructions within a command sequence changes during optimized compilation of the program.

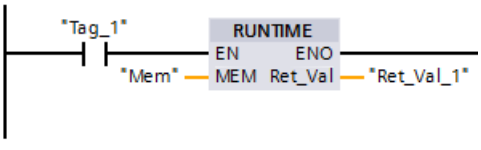
Table 8-155 Data types for the parameters

Parameter	Data type	Description
MEM	LReal	Starting poing of the runtime measurement
RET_VAL	LReal	Measured runtime in seconds

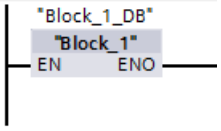
Example: RUNTIME instruction

The following example shows the use of the RUNTIME instruction to measure the execution time of a function block:

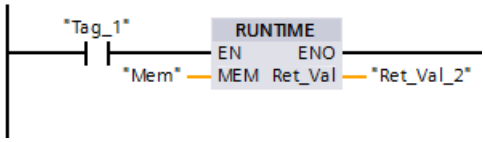
Network 1:



Network 2:



Network 3:



When the "Tag_1" operand in network 1 has the signal state "1", the RUNTIME instruction executes. The starting point for the runtime measurement is set with the first call of the instruction and buffered as reference for the second call of the instruction in the "Mem" operand.

The function block FB1 executes in network 2.

When the FB1 program block completes and the "Tag_1" operand has the signal state "1", the RUNTIME instruction in network 3 executes. The second call of the instruction calculates the runtime of the program block and writes the result to the output RET_VAL_2.

8.8.10 SCL program control statements

8.8.10.1 Overview of SCL program control statements

Structured Control Language (SCL) provides three types of program control statements for structuring your user program:

- Selective statements: A selective statement enables you to direct program execution into alternative sequences of statements.
- Loops: You can control loop execution using iteration statements. An iteration statement specifies which parts of a program should be iterated depending on certain conditions.
- Program jumps: A program jump means an immediate jump to a specified jump destination and therefore to a different statement within the same block.

These program control statements use the syntax of the PASCAL programming language.

Table 8-156 Types of SCL program control statements

Program control statement		Description
Selective	IF-THEN statement (Page 295)	Enables you to direct program execution into one of two alternative branches, depending on a condition being TRUE or FALSE
	CASE statement (Page 296)	Enables the selective execution into 1 of *n* alternative branches, based on the value of a variable
Loop	FOR statement (Page 297)	Repeats a sequence of statements for as long as the control variable remains within the specified value range
	WHILE-DO statement (Page 298)	Repeats a sequence of statements while an execution condition continues to be satisfied
	REPEAT-UNTIL statement (Page 299)	Repeats a sequence of statements until a terminate condition is met

Program control statement		Description
Program jump	CONTINUE statement (Page 300)	Stops the execution of the current loop iteration
	EXIT statement (Page 300)	Exits a loop at any point regardless of whether the terminate condition is satisfied or not
	GOTO statement (Page 301)	Causes the program to jump immediately to a specified label
	RETURN statement (Page 301)	Causes the program to exit the block currently being executed and to return to the calling block

8.8.10.2 IF-THEN statement

The IF-THEN statement is a conditional statement that controls program flow by executing a group of statements, based on the evaluation of a Bool value of a logical expression. You can also use brackets to nest or structure the execution of multiple IF-THEN statements.

Table 8-157 Elements of the IF-THEN statement

SCL	Description

`IF "condition" THEN`
`  statement_A;`
`  statement_B;`
`  statement_C;`
`  ;`
	If "condition" is TRUE or 1, then execute the following statements until encountering the END_IF statement.
	If "condition" is FALSE or 0, then skip to END_IF statement (unless the program includes optional ELSIF or ELSE statements).

`[ELSIF "condition-n" THEN`
`  statement_N;`
`  ;]`
	The optional ELSEIF[1] statement provides additional conditions to be evaluated. For example: If "condition" in the IF-THEN statement is FALSE, then the program evaluates "condition-n". If "condition-n" is TRUE, then execute "statement_N".

`[ELSE`
`  statement_X;`
`  ;]`
	The optional ELSE statement provides statements to be executed when the "condition" of the IF-THEN statement is FALSE.

`END_IF;`	The END_IF statement terminates the IF-THEN instruction.

[1] You can include multiple ELSIF statements within one IF-THEN statement.

Table 8-158 Variables for the IF-THEN statement

Variables	Description
"condition"	Required. The logical expression is either TRUE (1) or FALSE (0).
"statement_A"	Optional. One or more statements to be executed when "condition" is TRUE.
"condition-n"	Optional. The logical expression to be evaluated by the optional ELSIF statement.
"statement_N"	Optional. One or more statements to be executed when "condition-n" of the ELSIF statement is TRUE.
"statement_X"	Optional. One or more statements to be executed when "condition" of the IF-THEN statement is FALSE.

An IF statement is executed according to the following rules:

- The first sequence of statements whose logical expression = TRUE is executed. The remaining sequences of statements are not executed.
- If no Boolean expression = TRUE, the sequence of statements introduced by ELSE is executed (or no sequence of statements if the ELSE branch does not exist).
- Any number of ELSIF statements can exist.

Note

Using one or more ELSIF branches has the advantage that the logical expressions following a valid expression are no longer evaluated in contrast to a sequence of IF statements. The runtime of a program can therefore be reduced.

8.8.10.3 CASE statement

Table 8-159 Elements of the CASE statement

SCL	Description

`CASE "Test_Value" OF`
`  "ValueList": Statement[; Statement, ...]`
`  "ValueList": Statement[; Statement, ...]`
`[ELSE`
`Else-statement[; Else-statement, ...]]`
`END_CASE;`
	The CASE statement executes one of several groups of statements, depending on the value of an expression.

Table 8-160 Parameters

Parameter	Description
"Test_Value"	Required. Any numeric expression of data type Int
"ValueList"	Required. A single value or a comma-separated list of values or ranges of values. (Use two periods to define a range of values: 2..8) The following example illustrates the different variants of the value list: 1: Statement_A; 2, 4: Statement _B; 3, 5..7,9: Statement _C;
Statement	Required. One or more statements that are executed when "Test_Value" matches any value in the value list
Else-statement	Optional. One or more statements that are executed if no match with a value of the "ValueList" stated matches

The CASE statement is executed according to the following rules:

- The Test_value expression must return a value of the type Int.
- When a CASE statement is processed, the program checks whether the value of the Test_value expression is contained within a specified list of values. If a match is found, the statement component assigned to the list is executed.
- If no match is found, the program section following ELSE is executed or no statement is executed if the ELSE branch does not exist.

Example: Nested CASE statements

CASE statements can be nested. Each nested case statement must have an associated END_CASE statement.

```
CASE "var1" OF
      1 : #var2 := 'A';
      2 : #var2 := 'B';
ELSE
      CASE "var3" OF

         65..90: #var2 := 'UpperCase';
         97..122: #var2 := 'LowerCase';
      ELSE

         #var2:= 'SpecialCharacter';
      END_CASE;
END_CASE;
```

8.8.10.4 FOR statement

Table 8-161 Elements of the FOR statement

SCL	Description

`FOR "control_variable" := "begin" TO "end"`
`[BY "increment"] DO`
`  statement;`
`  ;`
`END_FOR;`
	A FOR statement is used to repeat a sequence of statements as long as a control variable is within the specified range of values. The definition of a loop with FOR includes the specification of an initial and an end value. Both values must be the same type as the control variable.
	You can nest FOR loops. The END_FOR statement refers to the last executed FOR instruction.

Table 8-162 Parameters

Parameter	Description
"control_variable"	Required. An integer (Int or DInt) that serves as a loop counter
"begin"	Required. Simple expression that specifies the initial value of the control variables
"end"	Required. Simple expression that determines the final value of the control variables
"increment"	Optional. Amount by which a "control variable" is changed after each loop. The "increment" has the same data type as "control variable". If the "increment" value is not specified, then the value of the run tags will be increased by 1 after each loop. You cannot change "increment" during the execution of the FOR statement.

The FOR statement executes as follows:

- At the start of the loop, the control variable is set to the initial value (initial assignment) and each time the loop iterates, it is incremented by the specified increment (positive increment) or decremented (negative increment) until the final value is reached.
- Following each run through of the loop, the condition is checked (final value reached) to establish whether or not it is satisfied. If the end condition is not satisfied, the sequence of statements is executed again, otherwise the loop terminates and execution continues with the statement immediately following the loop.

Rules for formulating FOR statements:

- The control variable may only be of the data type Int or DInt.
- You can omit the statement BY [increment]. If no increment is specified, it is automatically assumed to be +1.

To end the loop regardless of the state of the "condition" expression, use the EXIT statement (Page 300). The EXIT statement executes the statement immediately following the END_FOR statement.

Use the CONTINUE statement (Page 300) to skip the subsequent statements of a FOR loop and to continue the loop with the examination of whether the condition is met for termination.

8.8.10.5 WHILE-DO statement

Table 8-163 WHILE statement

SCL	Description

`WHILE "condition" DO`
`  Statement;`
`  Statement;`
`  ...;`
`END_WHILE;`
	The WHILE statement performs a series of statements until a given condition is TRUE.
	You can nest WHILE loops. The END_WHILE statement refers to the last executed WHILE instruction.

Table 8-164 Parameters

Parameter	Description
"condition"	Required. A logical expression that evaluates to TRUE or FALSE. (A "null" condition is interpreted as FALSE.)
Statement	Optional. One or more statements that are executed until the condition evaluates to TRUE.

Note

The WHILE statement evaluates the state of "condition" before executing any of the statements. To execute the statements at least one time regardless of the state of "condition", use the REPEAT statement (Page 299).

The WHILE statement executes according to the following rules:

- Prior to each iteration of the loop body, the execution condition is evaluated.
- The loop body following DO iterates as long as the execution condition has the value TRUE.
- Once the value FALSE occurs, the loop is skipped and the statement following the loop is executed.

To end the loop regardless of the state of the "condition" expression, use the EXIT statement (Page 300). The EXIT statement executes the statement immediately following the END_WHILE statement.

Use the CONTINUE statement to skip the subsequent statements of a WHILE loop and to continue the loop with the examination of whether the condition is met for termination.

8.8.10.6 REPEAT-UNTIL statement

Table 8-165 REPEAT instruction

SCL	Description

`REPEAT`
`  Statement;`
`  ;`
`UNTIL "condition"`
`END_REPEAT;`
	The REPEAT statement executes a group of statements until a given condition is TRUE.
	You can nest REPEAT loops. The END_REPEAT statement always refers to the last executed Repeat instruction.

Table 8-166 Parameters

Parameter	Description
Statement	Optional. One or more statements that are executed until the condition is TRUE.
"condition"	Required. One or more expressions of the two following ways: A numeric expression or string expression that evaluates to TRUE or FALSE. A "null" condition is interpreted as FALSE.

Note

Before evaluating the state of "condition", the REPEAT statement executes the statements during the first iteration of the loop (even if "condition" is FALSE). To review the state of "condition" before executing the statements, use the WHILE statement (Page 298).

To end the loop regardless of the state of the "condition" expression, use the EXIT statement (Page 300). The EXIT statement executes the statement immediately following the END_REPEAT statement.

Use the CONTINUE statement (Page 300) to skip the subsequent statements of a REPEAT loop and to continue the loop with the examination of whether the condition is met for termination.

8.8.10.7 CONTINUE statement

Table 8-167 CONTINUE statement

SCL	Description

`CONTINUE`
`Statement;`
`;`
	The CONTINUE statement skips the subsequent statements of a program loop (FOR, WHILE, REPEAT) and continues the loop with the examination of whether the condition is met for termination. If this is not the case, the loop continues.

The CONTINUE statement executes according to the following rules:

- This statement immediately terminates execution of a loop body.
- Depending on whether the condition for repeating the loop is satisfied or not the body is executed again or the iteration statement is exited and the statement immediately following is executed.
- In a FOR statement, the control variable is incremented by the specified increment immediately after a CONTINUE statement.

Use the CONTINUE statement only within a loop. In nested loops CONTINUE always refers to the loop that includes it immediately. CONTINUE is typically used in conjunction with an IF statement.

If the loop is to exit regardless of the termination test, use the EXIT statement.

Example: CONTINUE statement

The following example shows the use of the CONTINUE statement to avoid a division-by-0 error when calculating the percentage of a value:

```
FOR i := 0 TO 10 DO
IF value[i] = 0 THEN CONTINUE; END_IF;
    p := part / value[i] * 100;
    s := INT_TO_STRING(p);
    percent := CONCAT(IN1:=s, IN2:="%");
END_FOR;
```

8.8.10.8 EXIT statement

Table 8-168 EXIT instruction

SCL	Description
`EXIT;`	An EXIT statement is used to exit a loop (FOR, WHILE or REPEAT) at any point, regardless of whether the terminate condition is satisfied.

The EXIT statement executes according to the following rules:

- This statement causes the repetition statement immediately surrounding the exit statement to be exited immediately.
- Execution of the program is continued after the end of the loop (for example after END_FOR).

Use the EXIT statement within a loop. In nested loops, the EXIT statement returns the processing to the next higher nesting level.

Example: EXIT statement

```
FOR i := 0 TO 10 DO
```

```
CASE value[i, 0] OF
    1..10: value [i, 1]:="A";
    11..40: value [i, 1]:="B";
    41..100: value [i, 1]:="C";
ELSE
EXIT;
END_CASE;
END_FOR;
```

8.8.10.9 GOTO statement

Table 8-169 GOTO statement

SCL	Description

`GOTO JumpLabel;`
`Statement;`
`... ;`
`JumpLabel: Statement;`
	The GOTO statement skips over statements by jumping to a label in the same block.
	The jump label ("JumpLabel") and the GOTO statement must be in the same block. The name of a jump label can only be assigned once within a block. Each jump label can be the target of several GOTO statements.

It is not possible to jump to a loop section (FOR, WHILE or REPEAT). It is possible to jump from within a loop.

Example: GOTO statement

In the following example: Depending on the value of the "Tag_value" operand, the execution of the program resumes at the point defined by the corresponding jump label. If "Tag_value" equals 2, the program execution resumes at the jump label "MyLabel2" and skips "MyLabel1".

```
CASE "Tag_value" OF
1 : GOTO MyLabel1;
2 : GOTO MyLabel2;
ELSE GOTO MyLabel3;
END_CASE;
MyLabel1: "Tag_1" := 1;
MyLabel2: "Tag_2" := 1;
MyLabel3: "Tag_4" := 1;
```

8.8.10.10 RETURN statement

Table 8-170 RETURN instruction

SCL	Description
`RETURN;`	The Return instruction exits the code block being executed without conditions. Program execution returns to the calling block or to the operating system (when exiting an OB).

Example: RETURN instruction:

```
IF "Error" <> 0 THEN
RETURN;
END_IF;
```

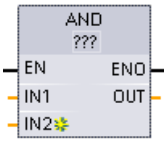
Note

After executing the last instruction, the code block automatically returns to the calling block. Do not insert a RETURN instruction at the end of the code block.

8.9 Word logic operations

8.9.1 AND, OR, and XOR logic operation instructions

Table 8-171 AND, OR, and XOR logic operation instructions

LAD / FBD	SCL	Description
	`out := in1 AND in2;`	AND: Logical AND
	`out := in1 OR in2;`	OR: Logical OR
	`out := in1 XOR in2;`	XOR: Logical EXCLUSIVE OR

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

To add an input, click the "Create" icon or right-click on an input stub for one of the existing IN parameters and select the "Insert input" command.

To remove an input, right-click on an input stub for one of the existing IN parameters (when there are more than the original two inputs) and select the "Delete" command.

Table 8-172 Data types for the parameters

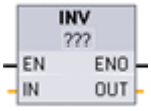
Parameter	Data type	Description
IN1, IN2	Byte, Word, DWord	Logical inputs
OUT	Byte, Word, DWord	Logical output

[1] The data type selection sets parameters IN1, IN2, and OUT to the same data type.

The corresponding bit values of IN1 and IN2 are combined to produce a binary logic result at parameter OUT. ENO is always TRUE following the execution of these instructions.

8.9.2 INV (Create ones complement)

Table 8-173 INV instruction

LAD / FBD	SCL	Description
	`Not available`	Calculates the binary one's complement of the parameter IN. The one's complement is formed by inverting each bit value of the IN parameter (changing each 0 to 1 and each 1 to 0). ENO is always TRUE following the execution of this instruction.

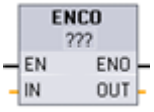
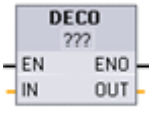
[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-174 Data types for the parameters

Parameter	Data type	Description
IN	SInt, Int, DInt, USInt, UInt, UDInt, Byte, Word, DWord	Data element to invert
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Byte, Word, DWord	Inverted output

8.9.3 DECO (Decode) and ENCO (Encode) instructions

Table 8-175 ENCO and DECO instruction

LAD / FBD	SCL	Description
	`out := ENCO(_in_);`	Encodes a bit pattern to a binary number The ENCO instruction converts parameter IN to the binary number corresponding to the bit position of the least-significant set bit of parameter IN and returns the result to parameter OUT. If parameter IN is either 0000 0001 or 0000 0000, then a value of 0 is returned to parameter OUT. If the parameter IN value is 0000 0000, then ENO is set to FALSE.
	`out := DECO(_in_);`	Decodes a binary number to a bit pattern The DECO instruction decodes a binary number from parameter IN, by setting the corresponding bit position in parameter OUT to a 1 (all other bits are set to 0). ENO is always TRUE following execution of the DECO instruction. Note: The default data type for the DECO instruction is DWORD. In SCL, change the instruction name to DECO_BYTE or DECO_WORD to decode a byte or word value, and assign to a byte or word tag or address.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-176 Data types for the parameters

Parameter	Data type	Description
IN	ENCO: Byte, Word, DWord DECO: UInt	ENCO: Bit pattern to encode DECO: Value to decode
OUT	ENCO: Int DECO: Byte, Word, DWord	ENCO: Encoded value DECO: Decoded bit pattern

Table 8-177 ENO status

ENO	Condition	Result (OUT)
1	No error	Valid bit number
0	IN is zero	OUT is set to zero

The DECO parameter OUT data type selection of a Byte, Word, or DWord restricts the useful range of parameter IN. If the value of parameter IN exceeds the useful range, then a modulo operation is performed to extract the least significant bits shown below.

DECO parameter IN range:

- 3 bits (values 0-7) IN are used to set 1 bit position in a Byte OUT
- 4-bits (values 0-15) IN are used to set 1 bit position in a Word OUT
- 5 bits (values 0-31) IN are used to set 1 bit position in a DWord OUT

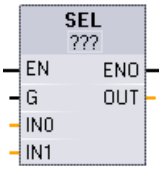
Table 8-178 Examples

DECO IN value			DECO OUT value (Decode single bit position)
Byte OUT 8 bits	Min. IN	0	00000001
	Max. IN	7	10000000
Word OUT 16 bits	Min. IN	0	0000000000000001
	Max. IN	15	1000000000000000
DWord OUT 32 bits	Min. IN	0	00000000000000000000000000000001
	Max. IN	31	10000000000000000000000000000000

8.9.4 SEL (Select), MUX (Multiplex), and DEMUX (Demultiplex) instructions

Table 8-179 SEL (select) instruction

LAD / FBD	SCL	Description

	`out := SEL(`
	`g:=_bool_in,`
	`in0:-_variant_in,`
	`in1:=_variant_in);`
		SEL assigns one of two input values to parameter OUT, depending on the parameter G value.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

Table 8-180 Data types for the SEL instruction

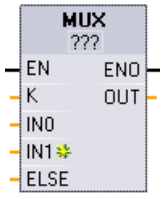
Parameter	Data type [1]	Description
G	Bool	• 0 selects IN0 • 1 selects IN1
IN0, IN1	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Inputs
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Output

[1] Input variables and the output variable must be of the same data type.

Condition codes: ENO is always TRUE following execution of the SEL instruction.

Table 8-181 MUX (multiplex) instruction

LAD / FBD	SCL	Description

	`out := MUX(`
	`  k:=_unit_in,`
	`  in1:=variant_in,`
	`  in2:=variant_in,`
	`  [...in32:=variant_in`
	`,]`
	`  inelse:=variant_in);`
		MUX copies one of many input values to parameter OUT, depending on the parameter K value. If the parameter K value exceeds (IN*n* - 1), then the parameter ELSE value is copied to parameter OUT.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

To add an input, click the "Create" icon or right-click on an input stub for one of the existing IN parameters and select the "Insert input" command.

To remove an input, right-click on an input stub for one of the existing IN parameters (when there are more than the original two inputs) and select the "Delete" command.

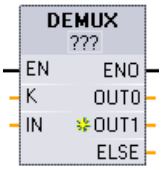
Table 8-182 Data types for the MUX instruction

Parameter	Data type	Description
K	UInt	• 0 selects IN1 • 1 selects IN2 • *n* selects IN*n*
IN0, IN1, .. INn	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Inputs
ELSE	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Input substitute value (optional)
OUT	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Output

[1] Input variables and the output variable must be of the same data type.

Table 8-183 DEMUX (Demultiplex) instruction

LAD / FBD	SCL	Description

	`DEMUX(`
	`k:=_unit_in,`
	`in:=variant_in,`
	`out1:=variant_in,`
	`out2:=variant_in,`
	`[...out32:=variant_in,]`
	`outelse:=variant_in)`
	`;`
		DEMUX copies the value of the location assigned to parameter IN to one of many outputs. The value of the K parameter selects which output selected as the destination of the IN value. If the value of K is greater than the number (OUT*n* - 1) then the IN value is copied to location assigned to the ELSE parameter.

[1] For LAD and FBD: Click the "???" and select a data type from the drop-down menu.

To add an output, click the "Create" icon or right-click on an output stub for one of the existing OUT parameters and select the "Insert output" command.

To remove an output, right-click on an output stub for one of the existing OUT parameters (when there are more than the original two outputs) and select the "Delete" command.

Table 8-184 Data types for the DEMUX instruction

Parameter	Data type [1]	Description
K	UInt	Selector value: • 0 selects OUT1 • 1 selects OUT2 • n selects OUTn
IN	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Input
OUT0, OUT1, .. OUTn	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Outputs
ELSE	SInt, Int, DInt, USInt, UInt, UDInt, Real, LReal, Byte, Word, DWord, Time, Date, TOD, Char, WChar	Substitute output when K is greater than (OUTn - 1)

[1] The input variable and the output variables must be of the same data type.

Table 8-185 ENO status for the MUX and DEMUX instructions

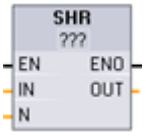
ENO	Condition	Result OUT
1	No error	MUX: Selected IN value is copied to OUT DEMUX: IN value is copied to selected OUT
0	MUX: K is greater than the number of inputs -1	• No ELSE provided: OUT is unchanged, • ELSE provided, ELSE value assigned to OUT
	DEMUX: K is greater than the number of outputs -1	• No ELSE provided: outputs are unchanged, • ELSE provided, IN value copied to ELSE

8.10 Shift and rotate

8.10.1 SHR (Shift right) and SHL (Shift left) instructions

Table 8-186 SHR and SHL instructions

LAD / FBD	SCL	Description

	`out := SHR(`
	`  in:=_variant_in_,`
	`  n:=_uint_in);`
	`out := SHL(`
	`  in:=_variant_in_,`
	`  n:=_uint_in);`
		Use the shift instructions (SHL and SHR) to shift the bit pattern of parameter IN. The result is assigned to parameter OUT. Parameter N specifies the number of bit positions shifted:
		• SHR: Shift bit pattern right
		• SHL: Shift bit pattern left

[1] For LAD and FBD: Click the "???" and select the data types from the drop-down menu.

Table 8-187 Data types for the parameters

Parameter	Data type	Description
IN	Integers	Bit pattern to shift
N	USInt, UDint	Number of bit positions to shift
OUT	Integers	Bit pattern after shift operation

- For N=0, no shift occurs. The IN value is assigned to OUT.
- Zeros are shifted into the bit positions emptied by the shift operation.
- If the number of positions to shift (N) exceeds the number of bits in the target value (8 for Byte, 16 for Word, 32 for DWord), then all original bit values will be shifted out and replaced with zeros (zero is assigned to OUT).
- ENO is always TRUE for the shift operations.

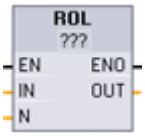
Table 8-188 Example: SHL for Word data

Shift the bits of a Word to the left by inserting zeroes from the right (N = 1)			
IN	1110 0010 1010 1101	OUT value before first shift:	1110 0010 1010 1101
		After first shift left:	1100 0101 0101 1010
		After second shift left:	1000 1010 1011 0100
		After third shift left:	0001 0101 0110 1000

8.10.2 ROR (Rotate right) and ROL (Rotate left) instructions

Table 8-189 ROR and ROL instructions

LAD / FBD	SCL	Description

	`out := ROL(`
	`  in:=_variant_in_,`
	`  n:=_uint_in);`
	`out := ROR(`
	`  in:=_variant_in_,`
	`  n:=_uint_in);`
		Use the rotate instructions (ROR and ROL) to rotate the bit pattern of parameter IN. The result is assigned to parameter OUT. Parameter N defines the number of bit positions rotated.
		• ROR: Rotate bit pattern right
		• ROL: Rotate bit pattern left

[1] For LAD and FBD: Click the "???" and select the data types from the drop-down menu.

Table 8-190 Data types for the parameters

Parameter	Data type	Description
IN	Integers	Bit pattern to rotate
N	USInt, UDint	Number of bit positions to rotate
OUT	Integers	Bit pattern after rotate operation

- For N=0, no rotate occurs. The IN value is assigned to OUT.
- Bit data rotated out one side of the target value is rotated into the other side of the target value, so no original bit values are lost.
- If the number of bit positions to rotate (N) exceeds the number of bits in the target value (8 for Byte, 16 for Word, 32 for DWord), then the rotation is still performed.
- ENO is always TRUE following execution of the rotate instructions.

Table 8-191 Example: ROR for Word data

Rotate bits out the right -side into the left -side (N = 1)			
IN	0100 0000 0000 0001	OUT value before first rotate:	0100 0000 0000 0001
		After first rotate right:	1010 0000 0000 0000
		After second rotate right:	0101 0000 0000 0000